Transactions on Accessible Computing

Multilingual Eye Gaze Controlled Adaptive Virtual Keyboard for Users with SSMI

Journal:	*Transactions on Accessible Computing*
Manuscript ID	TACCESS-06-21-0555
Manuscript Type:	Adaptive Inclusive AR/VR Systems Special Issue
Date Submitted by the Author:	23-Jun-2021
Complete List of Authors:	DV, JeevithaShree; Indian Institute of Science, Centre for Product Design and Manufacturing Jain, Puneeth; Indian Institute of Science Saluja, KamalPreet; Indian Institute of Science Biswas, Pradipta; Indian Institute of Science,
Keywords:	Severe Speech and Motor Impairment, Multilingual Virtual Keyboard, Genetic Algorithm, Adaptive Dwell Time, Markov Model, Eye Gaze Controlled Virtual Keyboard

SCHOLARONE™
Manuscripts

Multilingual Eye Gaze Controlled Adaptive Virtual Keyboard for Users with SSMI

Users with Severe Speech and Motor Impairment (SSMI) often use a communication chart through their eye gaze or limited hand movement and care takers interpret their communication intent. There is already significant research conducted to automate this communication process through electronic means. Developing electronic user interface and interaction techniques for users with SSMI poses significant challenges as research on their ocular parameters found that, such users suffer from Nystagmus and Strabismus limiting number of elements on a computer screen. This paper presents an eye gaze controlled multilingual virtual keyboard which has an optimized layout through a genetic algorithm and adaptive dwell time based on a Markov model-based algorithm. The virtual keyboard maintains a minimum inter-element spacing among keys and can automatically adapt to reduce eye gaze movement distance and dwell time for selection and help users with SSMI type better without any intervention of an assistant. The eye gaze controlled multilingual virtual keyboard is designed and developed for English and Kannada languages, respectively. With our proposed English virtual keyboard, the overall typing speed was 16.9 lpm (letters per minute) for able-bodied users and 6.6 lpm for users with SSMI without using any word completion or prediction features. Further, we noted a reduction in 13% and 6.05% reduction in total eye gaze movement distance in optimized layout than orderly arranged layout for English and Kannada languages, respectively. A case study with an elderly participant with SSMI found a typing speed of 2.70 wpm (words per minute) and 14.88 lpm (letters per minute) without using any word completion or prediction features after 4 months of practice. The proposed Kannada virtual keyboard can be extended to other Indian languages by just replacing their corresponding character set without any change in design of layout.

CCS CONCEPTS • Human Centered Computing
Additional Keywords and Phrases: Severe Speech and Motor Impairment, Multilingual Virtual Keyboard, Genetic Algorithm, Adaptive Dwell Time, Markov Model, Eye Gaze Controlled Virtual Keyboard

ACM Reference Format:

For Review Only

1 INTRODUCTION

Neuro-motor disabilities like ALS (Amyotrophic Lateral Sclerosis), Cerebral Palsy cause restriction of body movements and make conventional interaction mechanisms such as keyboard, mouse, or joystick more challenging. However, in many cases such motor disabilities do not reduce cognitive abilities. Various modes of communication used by people with SSMI are brain computer interfaces, head movements [1, 2, 3], and movements of larynx. Eye gaze interaction turns out to be a useful means of interaction for such users due to reduction of cost of COTS (Commercial-Off-The-Shelf) eye trackers and their non-invasive deployment [4, 5, 6].

This paper considers users with SSMI who can neither speak nor use sign language for communication. They often face difficulty in expressing themselves to rest of the world. They depend on special techniques like electronic user interfaces for communication, learning and many other daily activities. These special communication devices come under a broad heading of Augmentative and Alternative Communication (AAC) devices. AAC systems are forms of communication that supplement or enhance speech or writing. It includes communication aids other than speaking, such as: sign language, "yes, no" signals, gestures, picture board and computerized speech systems to compensate (either temporarily or permanently) for severe expressive communication disorders. People with SSMI often use a technique called eye pointing to communicate with outside world with the help of an assistant. An assistant holds a printed board (figure 1) in front of the person with SSMI and by analyzing his/her eye gaze manually, intentions of the person are interpreted by the assistant. This technique is often error prone and time consuming and depends on a single assistant/companion. We have tried to automate this process electronically using eye gaze trackers.

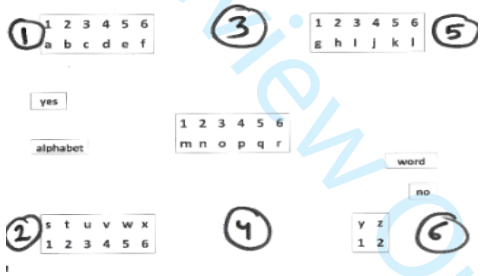


Figure 1: Non-Electronic Eye Pointing Chart

Eye tracking is the process of measuring either the point of gaze where one is looking or motion of an eye relative to head. An eye tracker is a device for measuring eye positions and eye movements [7]. This paper presents an eye gaze-controlled multilingual virtual keyboard that automatically adapts to reduce eye gaze movement distance and dwell time for selection. The virtual keyboard followed static and dynamic adaptation process as described in the AVANTI project [8]. The static adaptation process optimized the keyboard layout before start of user interaction and the dynamic adaptation process tracks users' interaction and reduces dwell time for commonly typed patterns. Before designing virtual keyboard, we undertook a pilot study to compare target users' interaction with different screen regions. In following sub sections, each process is described in detail. The main contributions of this paper are:

- Developing an application for comparing accessibility of different screen regions for an eye gaze-controlled interface

For Review Only

- Using Genetic Algorithm to optimize layout of a two-level eye gaze-controlled keyboard for English language and three-level eye gaze-controlled keyboard for Kannada language respectively
- Present an optimized gaze-controlled virtual keyboard layout for Kannada language which can be extended to other Indian languages like Hindi, Tamil, Telugu, Malayalam
- Using a Markov Model to optimize dwell time irrespective of the language model.

The paper is organized as follows: the next section presents a literature review on various optimization algorithms and eye gaze-controlled systems used for virtual keyboard design. Section 3 presents a detailed description of our target users with SSMI. Section 4 describes a pilot study to compare different screen regions with respect to an eye gaze-controlled interface followed by two sections 5 and 6 on static and dynamic adaptation techniques used in the proposed virtual keyboard. Section 7 presents discussion followed by concluding remarks in section 8.

2 LITERATURE REVIEW

An on-screen, virtual keyboard allows people with disabilities to interact through writing more easily and efficiently than with a traditional keyboard [9, 10]. The keys of a virtual keyboard are laid spatially on the computer screen. Users can compose text making single letter selections. Selection of keys from a virtual keyboard poses a significant problem for users with SSMI as they cannot use standard input devices like mouse. Scanning and access switches [11] requiring less motor control are commonly used. In recent time, Brain-Computer Interface that translates intent of a subject measured from brain signals directly into control command and eye tracking [1, 12, 13] which uses dwell time or blinking to directly manipulate user interface are explored. Dasher [14], FITALY [15], Metropolis [16], Swarachakra [17], pEYEs [18] are few examples of virtual keyboards using non-conventional modalities.

Layout Optimization.

It is important to design an optimized keyboard layout as it helps reduce typing error and increase typing speed for user. Thus, we surveyed articles from past research that focused on layout optimization and optimization of character locations on a soft keyboard. Lau [19] presented a virtual keyboard where keys are placed based on users' typing habit. They designed a customized virtual keyboard for users, who can only use one finger to type. They proposed a Genetic Algorithm based virtual keyboard. The author aimed to customize arrangement of keys as a 6 × 6 matrix to minimize typing cost using their finger as input. They compared performance based on typing cost between their proposed layout and existing QWERTY layout. They defined typing cost as the sum of finger movement and pressing effort. They noted that, their proposed keyboard can increase typing speed by 40% when compared to traditional keyboard. Li [15] proposed an optimized keyboard layout for users who can only operate a pointer with eyes. They used Fitts' Law and character transition frequencies of a large database. Three keyboard shapes were developed and were compared with FITALY and Metropolis [16] keyboards. They noted that, one of the proposed designs called ''YLAROF'' with a rectangular design and a symmetric layout with characters laid in ''I'' shape showed stable and consistent performance, when tested with 20 test cases. Patidar et al., [20] designed Quickpie, an interface for eye gaze-based text entry with only one depth layer of pie menu and selection border as selection method instead of dwell time. The authors maximized typing speed and minimized search time by removing dwell time. They used number of slices and width of characters region are crucial factors to achieve the same. Further, the authors aim to minimize effects of jittery

For Review Only

movements in the interface to minimize error rate. Prabhu [21] presented a virtual keyboard that can support multiple modes of input access with optimum layout based on frequency of occurrence of alphabets in an English text. They developed a method for optimal character placement for obtaining an appropriate scan time. Although they could not prove robustness of their layout, the authors compared the proposed layout against commonly used alphabetical layouts and noted that their layout had superior performance when compared to the rest. Various scan periods were evaluated to determine the most suitable scan times. Raynal [22] proposed a Genetic algorithm framework to optimize character locations on a soft keyboard. Their proposed method is independent of layout and language used. They devised a measure based on Mackenzie's model [23], to estimate performances of best characters in a given layout. To compare performance with FITALY, OPTI II or Metropolis keyboards, they applied their method to two different layouts in hexagonal and rectangular shapes. They noted that, in all configurations, the method proposed by them had best performance. They reported that keyboards produced by Genetic algorithm from OPTI II and Metropolis layout have highly promising results of 52% and 55% of improvement over QWERTY layout. Wagner [24] used Ant Colony Optimization algorithm to solve the problem of ergonomic optimization of distribution of characters over a keyboard layout. They developed a mathematical function for ergonomic evaluation criteria. They introduced a flexible keyboard model and coupled it with statistical analysis of concerned languages. They defined six ergonomic criteria mathematically. They designed an Ant Colony Optimization algorithm based on several different ant optimization approaches. Authors noted that, their proposed layout is better than manually optimized keyboards such as Dvorak and Marsan keyboards [24] in terms of typing speed and ease of memorization of arrangement.

From above literature, we noted that optimization of character placement is executed with a sufficiently large number of keys on screen. Our focused group users are individuals with SSMI and their limitation towards using an interface allows limited screen elements. Additionally, virtual keyboards typically require more frequent eye movements between keyboard and text display area [25]. Thus, our study focuses on developing an optimized keyboard design for placing appropriate characters in limited number of key slots on screen.

Virtual Keyboard in Different Languages.

In addition to designing virtual keyboard for English language, we aim to extend our proposed design to other Indian languages. Indian languages cover a wide range of linguistic variability when compared to English language. In this context, we looked into literature of Indian languages and their classification systems. Indian languages, spoken in states of India, are generally classified as belonging to the following families: Indo-European, Dravidian, Austroasiatic and Sino-Tibetan [26]. Of hundreds of languages spoken in India, 22 are mentioned in the constitution of India. Assamese, Bengali, Dogri, Gujarati, Hindi, Kashmiri, Konkani, Maithili, Marathi, Nepali, Oriya, Punjabi, Sanskrit, Sindhi and Urdu belong to the Indo-Aryan group of the Indo-European branch; Kannada, Malayalam, Tamil and Telugu belong to the Dravidian language family; and, of the three remaining languages, Manipuri, spoken in Manipur and Bodo spoken in North-Eastern India are usually classified as belonging to the Sino-Tibetan language family; and Santali is classified as a Austroasiatic language [26]. Most languages spoken in Northern part of India belong to the Indo-Aryan group of the Indo-European branch, while languages spoken in Southern part of India fall under the family of Dravidian languages. According to the Census of India, the largest group of Dravidian languages is South Dravidian, with almost 150 million speakers [27]. Tamil, Malayalam, and Kannada make up around 98% of speakers, with 75 million, 44 million and 37 million native speakers, respectively. The next largest spoken language is Telugu having 78 million

For Review Only

native speakers [27]. Hindi belonging to the Indo-Aryan family is considered the national language in India. Among the four popular Dravidian languages, we present our work towards the design of gaze-controlled virtual keyboard for Kannada language. Our proposed design can be extended to other Dravidian languages like Tamil, Telugu, and Malayalam and for Hindi languages. Additionally, we reviewed literature for design of existing multilingual virtual keyboards for Kannada, Tamil, Telugu, and Hindi languages. A few virtual keyboards exist to compose texts in Indian languages such as Google [28], Microsoft [29], gate2home [30], LooKeys [31], Lipik [32], MyLanguage [33], etc. These keyboards follow alphabetical ordering or Inscript (Indian script) mapping technique [33]. Moreover, the existing virtual keyboards occupy a larger screen area and therefore cannot be used for users with SSMI. Table 1 shows a detailed list of a few multilingual virtual keyboard designs. Table includes prospective relevant features and challenges we aim to consider for our design of gaze-controlled virtual keyboard for users with SSMI. It can be noted that Tamil language has lesser number of keys as they do not have Ottaksara. Rather they use a '.' (dot) above the consonant to depict any Ottaksara for a consonant [34].

Table 1: Detailed Description of Few Multilingual Virtual Keyboards- Hindi, Kannada, Tamil, Telugu Languages

Serial Number	Application Name	Language	Number of Selections Required for Ottaksara	Number of Screen Elements on a Single Screen	Prospective Relevant Features for Gaze-Controlled Virtual Keyboard
1	Indic Keyboard (Android) by Google LLC	Hindi	3	15 (Matra on first two rows)	• All letters and Matra are in a single layout • Not convenient to use
2	Indic Keyboard (Android) by Google LLC	Kannada	3	15 (Matra on first two rows)	• All letters and Matra are in a single layout • Cannot be used for gaze-controlled interfaces
3	Indic Keyboard (Android) by Google LLC	Tamil	2	12 (respective Matra are arranged on left side and letters on right of layout)	• Easy to differentiate between Matra and letters on a single layout • Can be used for gaze-controlled interfaces
4	Indic Keyboard (Android) by Google LLC	Telugu	3	16 (Matra arranged on left side, letters on right and Ottaksara in bottom of layout)	• Better to use when compared to other Telugu keyboards
5	Kannada Keys (IOS) by LE SANG	Kannada	3	28 letters on first screen with Matra and Ottaksara, when "Alt" pressed 11 letters appear	• "Alt" button to display only Matra and Ottaksara for corresponding letter
6	Kannada keyboard (IOS) by kannada keypad	Kannada	3	32 including Matra and Ottaksara	• Complex to use
7	Dot Hindi Hindi keyboard (Android) by FZ software	Hindi	3	16 (including Matra and Ottaksara mostly on left and few on right and bottom of layout)	• Complex to use • Arrangement of letters and Matra are random
8	Swarachakra (Android) by IDC, IIT Bombay	Hindi	3	10 (including Matra and	• Complicated to use for SSMI users

For Review Only

				Ottaksara)	
9	Telugu Keyboard Telugu Typing (Android) by Telugu Aps World	Telugu	3	33 Letters are distributed in four rows along with an upward arrow button When upward arrow button is pressed, five new rows appear which contains 30 letters with Matra, Ottaksara and numbers (up to 4 only)	• Poor design of layout • Can only adapt the concept of introducing upward arrow button to display Matra and Ottaksara
10	Tamil Keyboard (Android) by Arunk	Tamil	2	38 (all letters on one screen in 5 rows- Matra in the last row)	• Easy to differentiate between Matra and letters on a single layout • Can be used with gaze-controlled interfaces

Further, we explored literature that focused on layout optimization using various existing optimization algorithms. Meena [12, 13] proposed an optimized virtual keyboard layout for Hindi language. The interface is based on a menu wherein 10 commands provide access to type 88 different characters along with additional text editing commands. Five different keyboard layouts were presented and evaluated out of which two were found to be identified as best with eye tracking as input. Results showed that overall performance was high with respect to system usability and low workload. Malas [35] used Genetic algorithm to optimize design of Arabic keyboard layout for typing speed. They proposed a fitness function considering key distance, finger, and hand alteration. They considered frequency of individual letters and letter pairs. Authors claimed that their proposed keyboard layout is 35% faster than common layout. Khan [36] presented an optimized Urdu virtual keyboard layout. They used Monte Carlo Simulation with simulated annealing to optimize the layout. Layout was developed based on character frequency analysis of Urdu Corpus. To speed up text entry, the layout was augmented with word prediction list derived from Urdu corpus. Although study of performance with other keyboards was not tested, they evaluated the layout with 20 participants and noted that average text entry speed was 13.47 wpm with maximum speed achieved being 22.5 wpm. Joshi [17] developed a logically structured, open-source keyboard for text entry called Swarachakra. This keyboard works on Android platforms with touch as input. Layout of the keyboard is based on structure of the Indic scripts. The Swarachakra keyboard converts two or more taps on a typical keyboard into a single slide. The keyboard is available for 10 Indian languages (Hindi, Marathi, Gujarati, Telugu, Kannada, Malayalam, Odia, Punjabi, Bengali, and Konkani). Brouillette [37] explored use of multi-objective optimization and machine learning to create effective keyboards. Authors used Genetic algorithm to solve a multi-objective optimization problem and developed keyboards for two different languages, English, and Assamese. They reported that Assamese keyboard performed better than QWERTY layout.

As mentioned above, Indian languages cover a wide range of linguistic variability when compared to English language. We looked into literature of Indian languages to understand their classification systems, character set and phonology. Our focus lies in Dravidian languages like Kannada, Tamil, Telugu, Malayalam, and Hindi. We chose Kannada language to implement our proposed multilingual virtual keyboard. We reviewed literature for existing work in design of existing multilingual virtual keyboards for Kannada, Tamil, Telugu, and Hindi languages. We identified relevant features and challenges and proposed a virtual keyboard design which presently works for Kannada language and can work for other Indian languages like Hindi, Telugu, and Tamil.

For Review Only

Virtual Keyboard with Eye Tracking

Cecotti [38] elaborated interaction issues with eye tracking and presented a multimodal virtual keyboard. Cecotti's virtual keyboard is based on menu selection with eight main commands that allow users to spell 30 different characters and correct errors with a delete button. The system was evaluated with 18 adult participants in three conditions corresponding to three modalities: direct selection using a mouse, with an eye-tracker to point at desired command and a switch to select it, and with only eye tracker for command selection. They noted that, although selection through eye gaze reduces average typing speed, selection through mouse provides the best performance with respect to average typing speed. However, they found no difference in overall performance between mouse and eye tracker when compared to switch. Rajanna et al, [39] investigated how keyboard design, selection method, and motion in the field of view may impact using gaze-tracking in virtual reality. The main disadvantage of this study was the use of fixed dwell time of 550ms. Morimoto et al, [40] on the other hand introduced two new keyboard designs based on context switching for efficient and comfortable selection and expanding targets to facilitate control for reduced sized keyboards without the use of dwell time. Dasher [41, 15] system claimed that using their system with gaze-based method, increased the text entry speed by almost twice as high as other modes based on a longitudinal study that consisted of 10 sessions of 15 minutes of gaze writing over a period of one month without prior experience with gaze writing. They compared the above as a situation where a user with different range of abilities suddenly loses movement of other muscles and eyes become a completely new means of communication. Authors noted that, after 2.5 hours of practice, users were able to enter text at an average rate of 17.26 wpm, with top performer reaching 23.11 wpm. AltTyping [22] is a dwell-based method that allows user to adjust dwell time directly on the keyboard interface. MacKenzie and Zhang [23] used a language model to predict next three letters and proposed an algorithm that aims to help users drift their attention towards likely keys thereby reducing error rates and increasing typing speed. Pi and Shi [42] proposed a probabilistic model for gaze-based selection, which adjusted the dwell time based on probability of each letter selected in the past. The model automatically assigned shorter dwell times to more likely characters and longer dwell times to less likely characters using generative model for gaze in dwell-based typing and n-gram character model for letter prediction. Authors demonstrate that the proposed technique improved typing speed (12.39 wpm) without loss in accuracy. Majaranta [43] investigated adjusting dwell time for individual user and reported increase in typing speed when dwell time is user defined. Penkar [44] investigated different dwell time durations for different target sizes although putting screen elements at center of screen and found shorter dwell time and big target size as best combination. OptiDwell [45] system used multi-arm bandits, a reinforcement learning approach to adjust dwell time of keys. It uses the common principle of exploration to adjust dwell time setting and hence requires an extra button to identify Midas touch to decrease error. Cascading dwell gaze typing developed by Mott [46], decreases dwell time of likely keys while simultaneously increasing it for keys adjacent to probable keys. Reported average typing speed for people with ALS is 9.51 wpm which is evaluated on a single screen QWERTY keyboard. However, finding keys near to a particular key depends on design of the application and works best when there is symmetry.

Most previously described work was either not evaluated involving people with different range of abilities or with a group having less severe impairment compared to our target group of users with SSMI. Our target group presents new challenges in terms of size of screen elements, their positions on a computer screen and speed of interaction. The next section describes a set of 9 participants of our target user group followed by the proposed virtual keyboard layout.

For Review Only

3 DESCRIPTION OF USERS

Our end users were quadriplegic and were keen to learn operating computer. All trials and interactions with them were undertaken under observation by their care takers and all necessary permissions and ethical approvals were taken before undertaking user trials. We took help from their teachers, who are rehabilitation experts, to evaluate their physical conditions. According to Gross Motor Function Classification system (GMFCS), they were all at level 5 as they could not move without wheelchair. According to Manual Ability Classification System (MACS), some of them were at level 4 and rest were at level 5. A few of them could manage to move their hand to point to a non-electronic communication chart and others only relied on eye pointing. According to Communication Function Classification System (CFCS), all of them were at level 5 as they could not speak, could make only non-speech sound, and communicate only through non-electronic communication board. They did not have access to any commercially available scanning software. Their teachers and parents informed us that they were accustomed to use eye pointing with nonelectronic communication chart as all of them appear in examination using eye pointing to a non-electronic spelling chart. We have described more details on individual users in following paragraphs.

- *Participant A* | *14 years* | *(F)* |*NIOS (National Institute of Open Schooling Standard)-10th standard:* She understands logic easily. She has no finger isolation i.e. she cannot use hand to point something. She cannot press a switch with her hands and chin due to tremors (athetoid movement).
- *Participant B* | *20 years* | *(F)* | *NIOS-12th standard:* She takes time to understand things. She cannot hold things with her hand lacking finger isolation. She needs help of an assistant for keeping her head straight. She responds to yes/no by nodding her head. Her voice bottles up but can make some unclear sounds to draw attention. She is comfortable to use her head and chin.
- *Participant C* |*15 years* | *(M)* | *NIOS-10th standard:* He is cheerful and cooperative. He smiles and laughs when praised or does something well. He has rigid body and head movement. He responds by gentle blinks to say yes.
- *Participant D* | *06 years* | *(F)* | *STATE Syllabus- 3rd standard:* She cannot speak but responds to general talks by making voices. She is not so fluent in her hand movements but can use them to point something. She likes to interact with people by taking inputs from an assistant.
- *Participant E* | *06 years* | *(M)* | *NIOS 2nd standard:* He gets tensed easily. He is keen in listening to stories and loves to interact with people. He uses eyes, smile, and facial expressions to respond. He cannot move his hand.
- *Participant F* | *12 years* | *(F)* | *Prevocational Training:* She has inhibiting nature. She is shy and tries to speak with a low voice. She gets distracted easily. She has control of her head and hand movement. She always keeps her head down.
- *Participant G* | *7 years* | *(M)* | *NIOS 2nd standard:* He is a quick learner. He can communicate well with his eyes. He can point using his hands with good finger isolation. He can sight words on a spell chart comfortably.
- *Participant H* | *20 years* | *(F)* | *NIOS 12th standard:* She finds her interest in multimedia and can access computers well. She can use her hand to hold things like spoon. Her hands possess athetoid movement. She can speak with very poor clarity. She can use spell chart and QWERTY chart. She likes to interact with people and has a passion for making accessories. She can control head movement but has tremors.

- *Participant I | 23 years | (M) | NIOS 12th standard:* He is a joyful and obedient boy. He uses spectacles and is comfortable with eye tracking. He cannot speak properly. He responds to people by making sound and moving his head. He fears sudden sounds. He can ride his wheelchair himself.

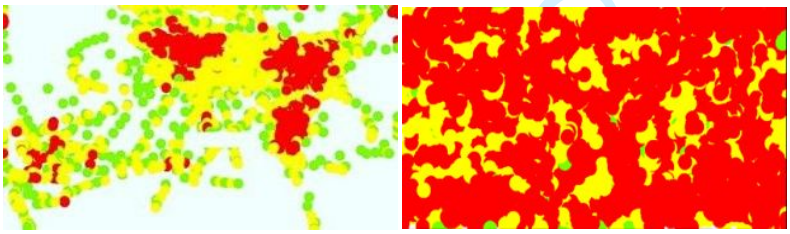
It may be noted that all users were keen to converse with others and be socially included. However, except for Participant H, no others can move their body parts voluntarily. All these users did not take part in all studies, we identified them by their codenames in subsequent studies.

4 PILOT STUDY

Screen elements on a graphical user interface are placed from left to right and top to bottom order. It is mostly assumed that users can access all regions of screen without difficulty. However, that assumption might not be true for eye gaze-controlled interfaces. Feit [47] reported able-bodied users can access top left corner of screen with more precision and accuracy than other regions while Jeevithashree [48] reported, users with SSMI find it difficult to access top left corner of screen using a gaze-controlled system.

The following study was designed to estimate spatial distribution of gaze points within a computer screen. We designed an application (hereafter referred as paint app) that generates a heat map of distribution of gaze points and then a graphics program tried to optimize the screen region which would be most comfortable for users to operate.

The study used similar concepts of Hornof's EyeDraw system [49] but unlike drawing a specific object, users were instructed to paint the whole screen. As users gaze at the screen, gaze tracker recorded gaze coordinates and we drew a green colored circle of 25 pixels radius centered at the gaze point. If a user gaze anywhere within the green circle more than 5 times, we re-painted the green circle in yellow and if a user gaze anywhere within the yellow circle more than 10 times, we repainted the yellow circle in red. After a user finished painting the screen, we got a heatmap showing regions where saccades launched more number of times than other region as well as regions where no saccades were launched. Figure 2(a) below shows such a heatmap created by a person with SSMI while Figure 2(b) shows the same for an able-bodied user.



(a) Heatmap created by user with SSMI (b) Heatmap created by able-bodied user

Figure 2. Heatmap Generated by Paint Application

After recording heatmap from each participant, we used the following graphics algorithm to identify most suitable area of screen. We initially measured average paint area for all participants using 'PaintArea algorithm' which is described below. It takes a set of heat map images as input and finds the percentage of paint area by participants using 'GetArea' function that takes any image as input and returns total number of pixels for red, yellow, and green area as output.

ALGORITHM 1: Function PaintArea(I)

Input: Input image of size MxN
Output: VArea - percentage of viewing area
1. [R, Y, G] = GetArea(I)
2. VArea = ((R+Y+G)/totalPixels) * 100

ALGORITHM 2: Function GetArea(inImg)

Input: Input image of size P×Q
Output: amount of red, yellow, and green pixels
do {
    1. fix threshold value to segment any colour areas
    2. use a binary mask to find the area of segmented regions
    3. sum all individual regions and store in r, y, g respectively for three colors
}
repeat (for all three colors)
return r, y, g

We used 'Optimum ScreenArea finding' algorithm to find optimal area out of full screen where users gazed frequently (i.e., maximum density of red, yellow, and green colors). It takes the set of heatmap images as input and generates cost of optimal area and its top left and bottom right corner as output. It starts the process by taking a window size of P×Q from top left corner of screen and continued by sliding the block along X-axis by p pixels, until it reaches the end. Then we slid it down along Y-axis by q pixels and followed the same procedure. We defined cost function to measure cost of each block. We noted weight of all colors and background area. Cost function was defined as $\Sigma c \in colours\ (weight * areac) - (weight * background)$. Pseudo code of our algorithm is as follows:

ALGORITHM 3: Function Optimal ScreenAreaFinding(I)

Input: Input image of size M×N
Output: cost (CF) and top left – bottom right coordinates
//$w1, w2, w3, w4$ indicate pre-defined weight of different color
cost = [ ]
cost1 = [ ]
for i = 0 to (N-Q)
for j = 0 to (M-P)
1. [R, Y, G] = GetArea(imcrop (I, [j, i, P, Q])) //Cropped image of size P×Q
2. Bg = total pixels in cropped image – (R+Y+G)
// Cost Function evaluation
3. CF = $w1$×R+$w2$×Y+$w3$×G-$w4$×Bg

For Review Only

4. cost = [CF j, i, j+P, i+Q]
5. cost1= cat (1, cost1, cost) //appending cost[ ] into cost1[]
6. j = j+p //shift p pixels along x axis

end

7. i = i+q //shift q pixels along y axis

End

Participants.

We collected data from all 9 users with SSMI described in section 2 and used above algorithms to find optimal area of screen.

Procedure.

Participants undertook the 9-point calibration routine provided by Tobii SDK. Then, we instructed them to paint the whole screen with their gaze using our painting app. We asked them to paint as much area as possible where they can gaze comfortably on screen. Each participant was asked to perform the painting task for at least 5 minutes.

Results.

We measured average paint area for all participants using 'PaintArea' algorithm. We considered 25 images and found that average area covered by painting was 59.91%. Then we considered a window having size of 1300×960 pixels, which is 60.18% of full screen size (1920×1080). We used 'Optimal ScreenArea Finding' algorithm to find optimal area and coordinates of it for all participants. We fixed weights for all colors as: $wred$ = 0.5, $wyellow$ = 0.3, $wgreen$ = 0.19, $wbackground$ = 0.01. Weights were set according to number of gaze points (for example red color had highest weight). The window was moved by 62 pixels along X axis and 60 pixels along Y axis, thus compared cost values of 33 rectangles for each image. Figure 3 shows optimum area marked with blue color bounding box in original image for a representative case. Average values of top left coordinates were (399, 38) [Stdev 208,54] and average values of bottom right coordinates were (1699,998) [Stdev 208, 54]. In only one case, optimum rectangle was anchored at top left corner of screen at (0,0) coordinate.

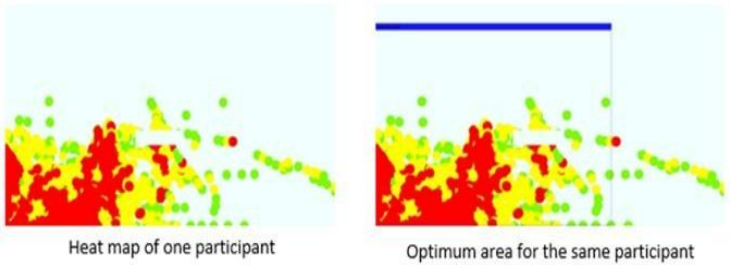


Figure 3. Result of Optimal Area Finding Algorithm

For Review Only

Discussion.

Average painted region was less than 100% indicating that participants found it difficult to gaze across all regions of screen. The screen was not painted following a regular pattern like left to right or top to bottom, rather generating random blobs of red and yellow regions. These random blobs of red regions indicated poor control of precise saccadic eye gaze movement. The study found that most participants could not paint top left corner of screen. Any screen element placed at top left corner will be difficult to select in a gaze-controlled interface by the present set of users. Finally, the algorithm designed for finding optimal screen area can be used to organize on screen elements.

It may be noted that we do not claim that users with ALS, Cerebral Palsy or in general with motor impairment cannot fixate attention at a specific region on screen. However, our study found that an analysis on fixation patterns can reveal areas of preference on screen and designing an interface without taking care of preferred screen regions may make it unusable.

5 STATIC ADAPTATION

Following the results from previous study, we designed an English virtual keyboard by placing all screen elements in middle region of screen avoiding placing any screen element at top left corner of screen. Our end users required a minimum target size and distance between targets for an eye gaze-controlled interface thus limiting the number of screen elements on screen. They could not use default Windows keyboard which shows all keys on a single screen. The user's non-electronic keyboard uses a two-level layout (figure 1) where the second level is uttered verbally.

We proposed a tree-based layout consisting of two levels (figure 4). First level consists of 9 buttons of letter groups. In second level each of the 9 buttons are expanded to individual letters respectively forming tree structure (figure 4). We considered grouping four letters into a single button as 9 buttons groups required smaller number of combinations as shown in Table 2.

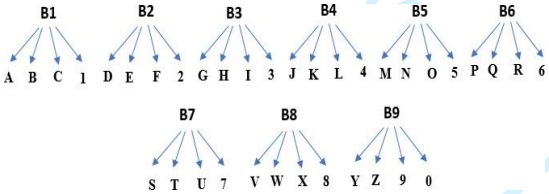


Figure 4. Tree-Based Structure

Alphabetic ordering is considered over QWERTY layout as our end users are aware of sequence from their preliminary grade of schooling. The keyboard layout has text area, letter buttons, space bar, auto word completion feature and delete button. Final design of our proposed English virtual keyboard layout is show in figure 5.

Table 2: Number of Combinations Generated on Grouping Letters in First Level

Screen-1 (n)	Screen-2 (k)	Total no. of combinations $(k!)^n$
8	5	4.29982E+16
9	4	2.64181E+12
7	6	1.00306E+20

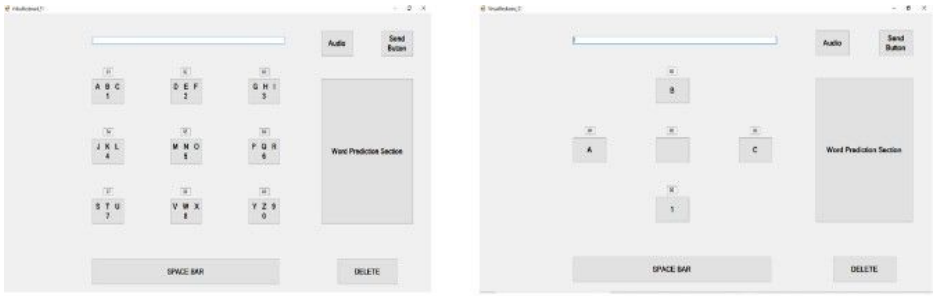
To operate

For Review Only

the keyboard, user must select a letter group in first screen and then the correct letter on second screen. Organization of keys in second screen decides the total amount of eye gaze movement and hence typing speed, considering visual search time will be constant after users become expert using keyboard. We have two choices of organizing keys in second screen with following advantages and disadvantages:

Orderly arrangement. We can arrange keys in clockwise or anticlockwise direction. The layout will be consistent, and reduces learning time, but will not minimize eye gaze movement distance.

Optimized Arrangement. We can optimize organization of keys on second screen for a sufficiently large corpus. Users may take longer to memorize key positions than an orderly arrangement, but once they turn



expert just like a new typist learns the standard QWERTY layout, the optimized layout will reduce total eye gaze movement distance and hence increase typing speed.

Figure 5. Two-Level Tree-Based Virtual Keyboard Interface for English language

Now, we can organize the four keys in second screen in 4! ways. As there are 9 buttons on first screen, there can be $(4!)^9$ combinations of keys for this keyboard layout. Karrenbauer [50] formulated a similar problem of placing n keys in n keyslots by formulating an integer programming problem using bigram frequency. However, it may be argued to use trigram or even higher order n-gram frequency that may further complicate formulating the objective function. As we are using two level layouts, number of possible solution space is higher than placing n keys in n keyslots.

We used Genetic algorithm to optimize arrangement of keys on second screen. We defined chromosomes by concatenating letters in second screen in clockwise pattern for all nine letter groups in first screen. For example, for screen layout of figure 5, the chromosome will be “abc1def2ghi3jkl4mno5pqr6stu7vwx8yz90”.

We started with four random arrangement of keys. Cost function is calculated as total eye gaze movement required to type Browns’ corpus [51]. We iterated the algorithm 5000 times. At each iteration, we undertook two crossover operations between the best and second-best strings and the best and third best string. We randomly mutate key arrangements for one of nine letter groups once in each iteration. Figure 6 shows total eye gaze movement distance to type Browns’ corpus for best orderly arrangement and optimized arrangements obtained through Genetic Algorithm. It may be noted that, there is a 13% reduction in total eye gaze movement distance in optimized layout than best orderly arranged layout. Following are results obtained:

- *Best Orderly Arrangement:* bc1aef2dhi3gkl4jno5mqr6ptu7swx8vz90y
- *Best Optimized Arrangement: ca1bdef2gih3lk4jm5no6prqts7u8vxw0y9z*

For Review Only

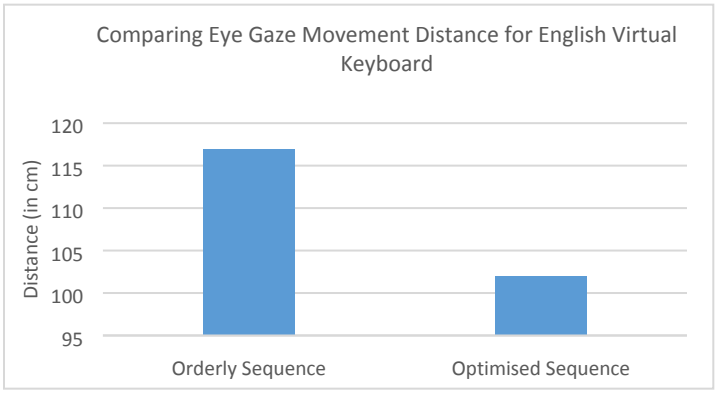


Figure 6. Comparing Eye Gaze Movement Distance between Orderly and Optimized Layout for English Virtual Keyboard

We may note that, keyboard layout can be optimized for any other corpus besides Brown's Corpus. Our definitions of gene and chromosome can be used for any Unicode character and rest of the Genetic Algorithm on string matching, crossover and mutation can work unchanged. Using this algorithm, we can optimize layout for other languages besides English with a corpus of chosen language.

5.1 Multilingual Virtual Keyboards

Features of character set in Indian languages are completely different from English language. Alphabet of English language consists of 26 characters [50] and Indian languages consists of 45 to 55 basic characters. In addition to this, Indian languages have approximately 150 complex characters possible. Therefore, deciding an optimal key arrangement leads to an exploration of a large solution space and appears to be a difficult problem in case of Indian languages. In English, alphabet can be composed in uppercase or lowercase, whereas complex characters (Ottaksara; Bengali Script; Devanagari; Telugu Script), inflexion (dependent vowel), and so on, are present in Indian languages [51]. Moreover, formation of a complex character requires at least three key presses or eye movements, which is more with respect to English character composition (one for a single small character and an extra shift to form capital character) [52]. Further, composing inflexed characters requires user to press two-character buttons, base character, and inflexion with respect to that base. As a result, user spends more time and effort to compose same-length characters in an Indian language when compared to English language. Since, occurrences of inflexed characters are high in Indian languages, placing each character as a single key slot is not possible as it eventually leads to a complex design of keyboard. Addressing the above-mentioned points with respect to Indian languages, in this section, we aim to discuss detailed approach to designing gaze-controlled virtual keyboard for Kannada language. From table 3, we may note that Kannada and Telugu languages have higher number of characters when compared to other Indian languages. On the other hand, Tamil language does not have conjunct consonant letters like other speeches [53] rather a simple "." (dot) is used above the consonant to create conjunct expression thereby having lesser number of characters. We have chosen Kannada language to design our proposed keyboard, as the over-all character set for Kannada language covers majority of characters therefore hitting the higher upper bound. However, our proposed design for Kannada language can be used for other Indian languages like Hindi, Tamil, Telugu, Malayalam and so on.

For Review Only

Table 3: List of Languages and Their Respective Number of Characters

Language	Number of Vowels	Number of Consonants	Number of Diacritics (Vowel+Consonant)	Number of Conjunct Consonants (Consonant+Consonant)
Kannada [53,54]	16	34	14	36
Hindi [55]	13	36	13	36
Tamil [56,57]	12	18	216	Nil (instead use ".")
Telugu [58]	15	36	15	36

5.1.1 Brief Introduction to Kannada Language.

The Kannada language is a Dravidian language spoken predominantly by people of Karnataka in South-Western India. Kannada language is written using Kannada script [59] from left-to-right. Like other Dravidian languages, Kannada language contains many more characters than English language. There are approximately 16 vowel characters (ಅ (a) to ಃ (aha)) including 2 partly vowels and partly consonants ಅಂ (am) and ಃ (aha); 34 consonants (ಕ (ka) to ಳ (lla)) including both structured (ಕ (ka) to ಮ (ma)) and unstructured (ಯ (ya) to ಳ (lla)) consonants respectively; 16 diacritic (Matra) for each consonant; more than 34 conjunct consonants (Ottaksara) which are formed by combination of a consonant and vowel; and a combination of two consonants respectively [60]; and 10 base numeric characters (0 to ೯) present in Kannada language (Table 4). We may note that, characters mentioned in table 4 are only base characters and many more characters (called complex and inflexed characters) can be composed with help of these base characters. Example for a Matra and Ottaksara in Kannada language [59] are given below:

- *Consonant + Vowel → Diacritic (Matra)*
  Example: ದ (da) + ಉ (u) → ದು (du)
- *Consonant + Consonant → Conjunct Consonant (Ottaksara)*
- *Example: ದ್ (d) + ದ (da) + ದ (da) → ದ್ದ (dda)*

Table 4. Kannada Language- Character Set: Vowels, Consonants, Diacritics, Conjunct

Character Type	Name	Description	Number of characters	Character Set
Vowel	Swara	Sound produced with a comparatively open configuration of vocal tract	16	ಅ(a) ಆ(AA) ಇ(e) ಈ(EE) ಉ(u) ಊ(UU) ಋ(ru) ೠ(rru) ಎ(ey) ಏ(eey) ಐ(I) ಒ(o) ಓ(OO) ಔ(ow) ಅಂ(am) ಅಃ(aha)
Consonant	Vyanjan	Speech sound that is articulated with complete or partial closure of vocal tract	34	ಕ(ka) ಖ(kha) ಗ(ga) ಘ(gha) ಙ(nga) ಚ(cha) ಛ(chha) ಜ(ja) ಝ(jha) ಞ(nya) ಟ(tta) ಠ(ttha) ಡ(dda) ಢ(ddha) ಣ(nna) ತ(tha) ಥ(tha) ದ(da) ಧ(dha) ನ(na) ಪ (pa) ಫ(pha) ಬ(ba) ಭ(bha) ಮ(ma)

For Review Only

				ಯ(ya) ರ(ra) ಲ(la) ವ(va) ಶ(sha) ಷ(Ssa) ಸ(sa) ಹ(ha) ಳ(lla)
Diacritic	Matra/Kagunita	Consonant + Vowel combination	16	Vowel diacritic with ಕ (ka): ಕ(ka) ಕಾ(kaa) ಕಿ(ki) ಕೀ (kii) ಕು(ku) ಕೂ(kuu) ಕೃ(kru) ಕೂ(kruu) ಕೆ(ke) ಕೇ(kee) ಕೈ (kai) ಕೊ(ko) ಕೋ (koo) ಕೌ(kou) ಕಂ(kum) ಕಃ(kaha)
Conjunct Consonant	Ottaksara	Letters representing consonants are combined to form ottakṣara with no intervening vowel	35 (corresponding to each consonant) and more	Some conjunct consonants are: ಕ್ಕ (kka) ಖ್ಖ(khha) ಗ್ಗ (gga) ಚ್ಚ(chha) ನ್ನ(nna) ಟ್ಟ (tta) ತ್ರ(tra) ಧ್ಯ(dhya) ದ್ದ(dda) ಬ್ಬ(bba) ಮ್ಮ(mma) ಲ್ಲ(lla) ಳ್ಳ(llah) ಹ್ಹ(hha) ಸ್ಪ(spa) ಸ್ಥ(stha) and so on
Numbers	Sankhyagalu	Basic numbers	10	೧(ondhu) ೨(eradu) ೩(muuru) ೪(nalakku) ೫(eydhu) ೬(aaru) ೭(yolu) ೮(yentu) ೯(ombattu) ೦(sonne)

5.1.2 Proposed Kannada Virtual Keyboard.

We propose to retain necessary conditions like avoiding usage of left part of screen to place elements, having limited elements on screen and so on, for Kannada language-based virtual keyboard in our design. Similar to English keyboard layout, we proposed a tree-based layout consisting of three levels as follows.

Level-1. First level consists of 9 keyslots of character groups, each when clicked expands to individual character keys in level 2; text entry bar; COMBINE (button with '+' sign) button for typing Ottaksara; space bar button; word prediction selection button; send button; and delete button. We have considered dividing vowels into two groups, consonants into six groups and numbers into one group respectively as shown in figure 7. Details of button grouping and their corresponding expansion in level-2 is given in table 5.

Level-2. Consists of 9 keyslots, each keyslot having a single character; text entry bar; word prediction selection button; COMBINE (button with '+' sign) button for typing Ottaksara; Matra (button named 'kagunita') button for selecting corresponding Matra for a character; back button; and delete button. Function of back button is to redirect user to shift to level-1 to select character groups. *Level-3.* This level is expanded only when a user selects 'kagunita' button for typing Matra for a character. This level consists of 17 keyslots each having single Matra for chosen character. When user clicks any of 17 keyslots/buttons, the interface redirects back to first screen (Level-1). From literature, we noted that most multilingual keyboards use three steps to select Ottaksara for a character except for Tamil language [57]. Due to limitation in space for placing screen elements, we proposed a three-level keyboard layout. Unlike usual keyboard layouts, placing all characters and Ottaksaras in a single page or displaying them in the form of pop up will make the interface cluttered and difficult to use leading to miss-clicks for users with SSMI [48].

For Review Only

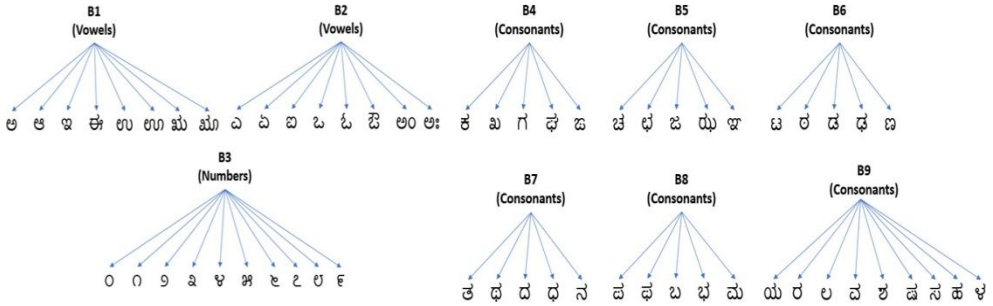


Figure 7. Tree Based Structure

Table 5: Grouping of Characters and Their Corresponding Expansion in Levels 1 and 2 Respectively

Character Type	Button Name	Number of Characters in Group	Button Groups on Level-1	Individual Expansion Buttons
Vowels	B1	8	ಅ(a) ಆ(AA) ಇ(e) ಈ(EE) ಉ(u) ಊ(UU) ಋ(ru) ೠ(rru)	B1, B2, B3, B4, B6, B7, B8, B9
Vowels & Number	B2	8	ಎ(ey) ಏ(eey) ಐ(I) ಒ(o) ಓ(OO) ಔ(ow) ಅಂ(am) ಅಃ(aha)	B1, B2, B3, B4, B6, B7, B8, B9
Numbers	B3	9	೧(ondhu) ೨(eradu) ೩(muuru) ೪(nalakku) ೫(eydhu) ೬(aaru) ೭(yolu) ೮(yentu) ೯(ombattu)	B1, B2, B3, B4, B5, B6, B7, B8, B9
Consonants	B4	5	ಕ(ka) ಖ(kha) ಗ(ga) ಘ(gha) ಙ(nga)	B1, B3, B5, B7, B9
Consonants	B5	5	ಚ(cha) ಛ(chha) ಜ(ja) ಝ(jha) ಞ(nya)	B1, B3, B5, B7, B9
Consonants	B6	5	ಟ(tta) ಠ(ttha) ಡ(dda) ಢ(ddha) ಣ(nna)	B1, B3, B5, B7, B9
Consonants	B7	5	ತ(tha) ಥ(tha) ದ(da) ಧ(dha) ನ(na)	B1, B3, B5, B7, B9
Consonants	B8	5	ಪ (pa) ಫ(pha) ಬ(ba) ಭ(bha) ಮ(ma)	B1, B3, B5, B7, B9
Consonants	B9	9	ಯ(ya) ರ(ra) ಲ(la) ವ(va) ಶ(sha) ಷ(Ssa) ಸ(sa) ಹ(ha) ಳ(lla)	B1, B2, B3, B4, B5, B6, B7, B8, B9

We followed alphabetical ordering of arrangement of characters as end users familiar with the language are aware of sequence from their preliminary grade of schooling. Selection of any character can be done as above

For Review Only

tree structure (Figure 7) of commands. When user selects one of nine keys of button groups at level-1, layout is transformed to level-2 consisting of nine separate keys where, each character is assigned separate key. Each of individual character buttons can be used to type desired base character. User can select desired base characters by traversing through level-1 and level-2, respectively. From level-2, user can redirect back to level-1 using 'back' button to select character groups. If user intends to use a Matra for a character in level-2, user must select desired character and then click 'kagunita' button which directs to level-3. As mentioned earlier, level-3 consists of 16 Matra for the chosen character which user can select. It may be noted that, once the desired Matra is selected, interface redirects back to first screen (Level-1). On the other hand, if user wants to type Ottaksara for any consonant, user must first select character group in level-1; select individual character from level-2; select COMBINE (button with '+' sign) button in level-2 to choose desired Ottaksara or click 'back' button to return to level-1 to choose desired character group to select Ottaksara. Let us consider design of Kannada virtual keyboard as shown in figure 9, and describe with examples typing a base character and working of two special keys 'kagunita' and COMBINE (button with '+' sign) buttons respectively:

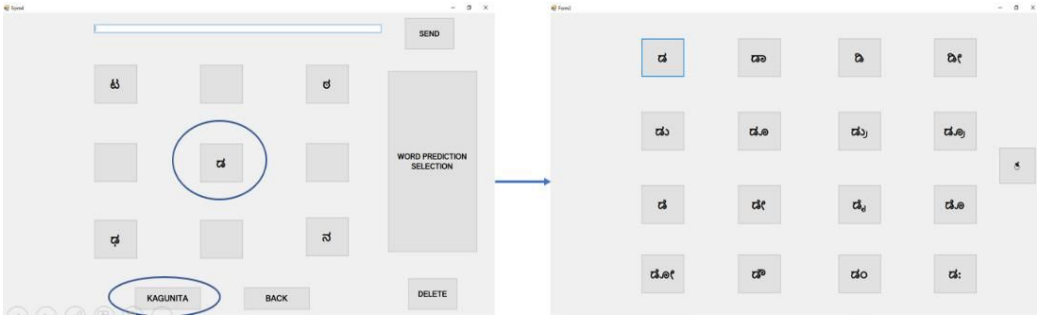
- *Typing a base character.* To type letter "ನ" (na), user must select key with character group having character ನ (na); layout then transforms to level-2 having each letter as separate keys. User must select desired letter key and press 'back' button.
- *Type a diacritic/ Matra.* To type letter "ಹು" (hu), user must select consonant element ಹ (ha) from level-2; click on 'kagunita' button which directs to level-3 consisting of 16 individual buttons (figure 8: Level-3) of all Matra for character ಹ (ha); chose desired Matra to form letter "ಹು" (hu).
- *Type a conjunct consonant/ Ottaksara.* To type letter "ಕ್ಕ" (kka), user must select consonant element ಕ (ka) from level-2; click on COMBINE (button with '+' sign) button and choose desired consonant from level-2. If consonant to form Ottaksara is not in same group, user must select 'back' button to go to level-1 and choose desired character group. User then selects character group containing letter ಕ (ka) from level-1 which expands to individual characters in level-2 as shown in Figure 8.

Consonant + Consonant → Conjunct Consonant (Ottaksara)



Example: ಕ (ka) + COMBINE (button with '+' sign) + ಕ (ka) → ಕ್ಕ(kka)

Left to Right: level-1(S1) and level-2 (S2) respectively (when user selects character group B6)



Left to Right: level-2 (S2) and level-3 (S3) respectively (when user selects 'kagunita' button after pressing button ಡ (Da))

Figure 8. Proposed Kannada Virtual Keyboard Layout

Figure 8 displays design of our proposed three level virtual keyboard for Kannada language. It may be noted that, we considered a scenario where user looks at text box every time, he/she types a character. Additionally, level-1, level-2 and level-3 are referred to as screens S1, S2 and S3, respectively. Our approach towards using proposed keyboard to type a word can be explained with an example as follows:

To type the word "ಕನ್ನಡ" (kannadi) which means mirror, user should operate proposed gaze-controlled keyboard as follows:

1. look for consonant group containing consonant ಕ (B4 – ಕ; ಖ; ಗ; ಘ; ಙ) in screen S1
2. on screen S2 character ಕ is selected. This forms base letter "ಕ"
3. selects 'back' button in screen S2
4. look at text box
5. look for consonant group containing consonant ನ (B7 – ತ; ಥ; ದ; ಧ; ನ) in screen S1
6. on screen S2 character ನ is selected
7. on screen S2, COMBINE (button with '+' sign) button is selected
8. on screen S2 character ನ is selected. This forms Ottaksara "ನ್ನ"
9. selects 'back' button
10. look at text box
11. look for consonant group containing consonant ಡ (B6 – ಟ; ಠ; ಡ; ಢ; ಣ) on screen S1
12. on screen S2 character ಡ is selected
13. on screen S2 'kagunita' (Matra) button is selected (directs to screen S3)
14. on screen S3 character ಡೆ is selected; screen S3 redirects back to screen S1. This forms letter "ಡೆ"
15. look at text box

Typing the word "ಕನ್ನಡೆ" (kannadi) is successfully completed. A short video demonstrating working of Kannada virtual keyboard can be found in the link: https://youtu.be/IGnYqVCHDY0?t=78.

Further, like the English keyboard layout optimization, we used Genetic algorithm to optimize arrangement of keys on second screen for Kannada keyboard. We defined chromosomes by concatenating letters in second screen in clockwise pattern for all nine letter groups in first screen. For example, for screen layout of figure 8, the chromosome will be "ಅಆಇಈಉಊಋಌಎಏಐಒಓಔಅಂಃ೧೨೩೪೫೬ಕಖಗಘಙಚಛಜಝಞಟಠಡಢಣತಥದಧನಪಫಬಭ

ಮಯರಲವಶಷಸಹಳ".

Since there are 9 keyslots on second screen, we started with nine random arrangement of keys. Cost function is calculated as total eye gaze movement required to type "Kannada monolingual sample dataset" corpus from Indian Language Technology Proliferation and Deployment Centre [61]. We iterated the algorithm 5000 times. At each iteration, we undertook two crossover operations between the best and second-best strings and the best and third best string. We randomly mutate key arrangements for one of nine letter groups once in each iteration. Figure 9 shows total eye gaze movement distance to type Kannada monolingual corpus for best orderly arrangement and optimized arrangements obtained through Genetic Algorithm. It may be noted that, there is a 6.04% reduction in total eye gaze movement distance in optimized layout than best orderly arranged layout. Following are results obtained:

- *Best Orderly Arrangement:*

  ಖುಅಈಆಖೂಿಊಔಅಂಎಒಏಃಐಔಒಓಏಎಔ

button click. All keyboard designs (Table 1) we came across, did not allow direct button selection to type a single Ottaksara. For example, to type character ಕ್ಷ (ksha) using any of existing Kannada keyboards, total number of key presses required was found to be 3 in following manner:

1. user must select character key ಕ (ka)
2. then select consonant ಕ್ (k)
3. finally select consonant ಷ (sha) for ottu

Sequence of 1, 2 and 3 i.e., ಕ + ಕ್ + ಷ will lead user to type Ottaksara ಕ್ಷ (ksha).

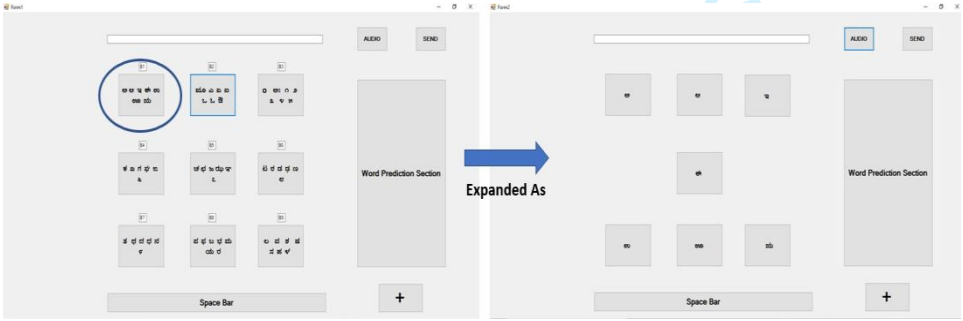
On the other hand, using our proposed keyboard mentioned in section 5.1.2., total number of key presses required to type a conjunct consonant character with COMBINE (button with '+' sign) button is 3. For example, if user wants to type the same character ಕ್ಷ (ksha), following steps are followed:

1. user must select character key ಕ (ka)
2. then select, COMBINE (button with '+' sign) button
3. finally select consonant ಷ (sha) for ottu

Sequence of 1, 2 and 3 i.e., ಕ + COMBINE button + ಷ will lead user to type Ottaksara ಕ್ಷ (ksha).

We may note that, organization of characters in keyslots on second screen (S2) decides total number of eye gaze movement distance and typing speed. Therefore, in this context, for understanding purpose, calculation for total number of key presses is considered by ignoring button group selection step. Although, total number of key presses remain same as existing keyboard designs, we claim that our proposed approach using COMBINE button to type Ottaksara is better. Unlike traditional way, user need not expand character ಕ (ka) as "ಕ್ (k) + ಕ (ka)" before selecting desired character for ottu (step 3 in above example). Instead, user can directly type Ottaksara as pronounced in a phonic way [62] using COMBINE (button with '+' sign) button as "ಕ (ka) + ಷ (sha) = ಕ್ಷ (ksha)". Phonics is a method of teaching children to read by linking sounds (phonemes) and symbols that represent them [62]. A good knowledge of phonics can help children develop active skill of writing [63].

Prior to our proposed three-level keyboard design (hereafter called as new layout), we designed a two-level Kannada layout (hereafter called as old layout) similar to new layout. The old layout consisted of only two-levels (Figure 10) and a single COMBINE (button with "+" sign) button was used for typing both Matra and Ottaksara (Figure 10) respectively as given in table 5. We may note that, any key press on second screen (level-2) redirects screen back to level-1.



Left- Level 1(S1), Right- Level 2 (S2)

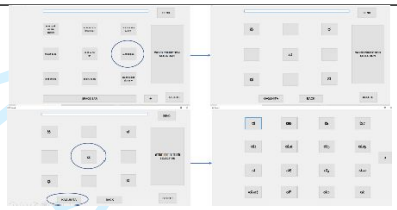
Figure 10. Two-Level Kannada Virtual Keyboard Layout (Old Layout)

Table 5: List of Languages and Their Respective Number of Characters

	Typing Matra Typing Ottaksara	**Typing Matra Typing Ottaksara**
Example considered	ಕ(ka) + ಇ(e) = ಕಿ(ki)	ಕ(ka) + ಕ(ka) = ಕ್ಕ (kka)
Steps followed	button group with ಕ (S1) → ಕ selected (S2) → COMBINE (button with '+' sign) button (S1) → button group with ಇ (S1) → ಇ selected (S2)	button group with ಕ (S1) → ಕ selected (S2) → COMBINE (button with '+' sign) button (S1) → button group with ಕ (S1) → ಕ selected (S2)
Total number key presses	5	5

We then compared old layout and new layout with existing virtual keyboard layout like Google Indic Kannada keyboard [64] and showed how our proposed layout takes lesser number of steps or key presses to type Matra and Ottaksara (Table 6) respectively. It may be noted that, old layout consists of two levels; new layout consists three levels and Google Indic Keyboard is a single level layout, respectively.

Table 6: Comparison Between Existing Google Indic Kannada Keyboard, Old Layout and Proposed New layout

	Old Layout [two-level keyboard]	**New Layout [three-level keyboard]**	**Existing Layout [single level layout]**
Layout Design	 (Reference Figure 10)	 (Reference Figure 8)	 Google Indic-Kannada Keyboard
Example considered to type Matra	ಕ(ka) + ಇ(e) = ಕಿ(ki)	ಕ(ka) + ಇ(e) = ಕಿ(ki)	ಕ(ka) + ಇ(e) = ಕಿ(ki)
Steps followed to type Matra	button group with ಕ (S1) → ಕ selected (S2) → COMBINE (button with '+' sign) button (S1) → button group with ಇ (S1) → ಇ selected (S2)	button group with ಕ (S1) → ಕ selected (S2) → 'kagunita' button (S2) → ಕಿ selected (S3)	select letter ಕ →ಕಿ selected (letters in green colour changes to matras of ಕ; rest of keys remain as is
Total number of key presses to type Matra	including selection of button groups: 5 excluding selection of button groups: 3	including selection of button groups: 4 excluding selection of button groups: 3	excluding selection of button groups: 2
Example considered to type Ottaksara	ಕ(ka) + ತ (ta) = ಕ್ತ (kta)	ಕ(ka) + ತ (ta) = ಕ್ತ (kta)	ಕ(ka) + ತ (ta) = ಕ್ತ (kta)
Steps	button group with ಕ (S1) →ಕ	button group with ಕ (S1) →ಕ	select letter ಕ → select

For Review Only

followed to type Ottaksara	selected (S2) → COMBINE (button with '+' sign) button (S1) → button group with ತ (S1) → ತ selected (S2)	selected (S2) → COMBINE (button with '+' sign) button (S2) → button group with ತ (Si) → ತ selected (Sj)	button ಕ್ → select letter ತ
Total number of key presses to type Ottaksara	including selection of button groups: 5; 5 if ottu is of same character, Example: ಕ್ಕ (kka)	including selection of button groups: 5; 4 if ottu is of same character, Example: ಕ್ಕ (kka)	excluding selection of button groups: 3
	excluding selection of button button groups: 3 groups: 3 and 3 if ottu is of same character, Example: ಕ್ಕ (kka)	excluding selection of button groups: 3 and 3 if ottu is of same character, Example: ಕ್ಕ (kka)	

From the above table (Table 6), total number of key presses required to type a Matra and Ottaksara using existing Google Indic keyboard is 2 and 3, respectively. On the other hand, total number of key presses (including selection of button groups) taken by a user to type Matra and Ottaksara with old layout is 5 and 5; new layout is 4 and 5 (and 4 if ottu is of same character as ಕ್ಕ (kka)) respectively. The Google Indic keyboard does not have required inter-button spacing necessary for our focused group of users with SSMI as stated in our earlier studies [48]. Thus, even when total number of key presses required to type using Google Indic keyboard (refer figure in table 6) is less when compared to new and old layout, users with SSMI, found it difficult to use the same. From this, we can say that our proposed three-level layout (new layout) takes lesser number of steps or key presses to type Matra and Ottaksara when compared old layout. Therefore, we can justify that, our proposed three-level layout (Figure 8) supports users with SSMI to type faster and with ease.

6 DYNAMIC ADAPTATION

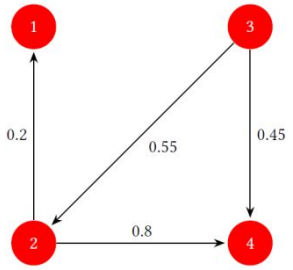
Static adaptation process fixes layout of keyboard and does not change it during user interaction. In a direct manipulation interface, a pointing task consists of two phases – pointing and selection. In this study, the optimized eye gaze-controlled keyboard layout aims to reduce eye gaze movement distance and pointing time. The selection was done by dwelling on a screen element. The dynamic adaptation proposes the following algorithm to reduce dwell time for commonly typed patterns.

Algorithm for Adaptive Dwell Time

Our algorithm considers every key/button as a Markovian state and assumes the movement of users' eye gaze between keys as a hop from one state to another. For example, if a user types a word 'RAT' on a QWERTY keyboard, user has moved from state 'R' to 'A' to 'T'. We assume moving to a state is only dependent on previous state and is independent of all past visited states. In other words, our model is a first order Markov model where all buttons are Markov states (IID's).

What is Markov Model?

In probability theory, a Markov model is a stochastic model used to model randomly changing system. It is assumed that future states depend only on current state, and not on events that occurred before it (that is, it assumes the Markov property). There are various forms of Markov models of which, the one we are using is called a Markov chain. Figure 11 shows an example of Markov chain having four states 1,2,3 and 4. Each arrow



For Review Only

depicts moving from one state to another while number on the edge shows the transition probability for that movement.

Figure 11. A Representation of Markov Chain

Working of Algorithm.

We construct an n×n transition matrix where n is equivalent to number of keys on screen. The matrix is called transition matrix since it contains information regarding transition probability of moving from one key to another (or one state to another). Each (i, j) element of matrix corresponds to conditional probability P (i | j), which is the probability of user clicking key 'i' after clicking key 'j'.

Initially we consider a movement from button j (state j) to any button i (state i) equally probable hence each button 'i' after button 'j' has equal dwell time. For example, initial transition matrix, T for n states (buttons) will be:

$$T = \begin{bmatrix} \frac{1}{n} & \frac{1}{n} & \cdots \\ \vdots & \ddots & \\ \frac{1}{n} & & \frac{1}{n} \end{bmatrix}$$

The maximum dwell time of any key is 1000ms (Dmax) and to avoid any unnecessary button click events (wrong selections or midas touch), we keep minimum dwell time to be 300ms (Dmin).

We follow an inverse relationship between these transition probabilities and dwell time of keys. Dwell time, Di (dwell time of key i) and conditional probabilities P (i | j) follow the following relationship:

$$\mathrm{P}(i \mid j) = \frac{2}{1 + \exp\left(\frac{D_i}{D_{min}} - 1\right)} \tag{1}$$

$$\Longrightarrow D_i = D_{min}\left[\log\left(\frac{2}{\mathrm{P}(i \mid j)} - 1\right) + 1\right]$$

To elaborate Equation 1, one should interpret for some fixed i and j, if P (i | j) = 1 and it is highly probable that user would dwell on key i after clicking key j. In other words, key i should have minimum dwell time Dmin as compared to all keys. This becomes more evident by substituting P (i | j) = 1 in above equation.

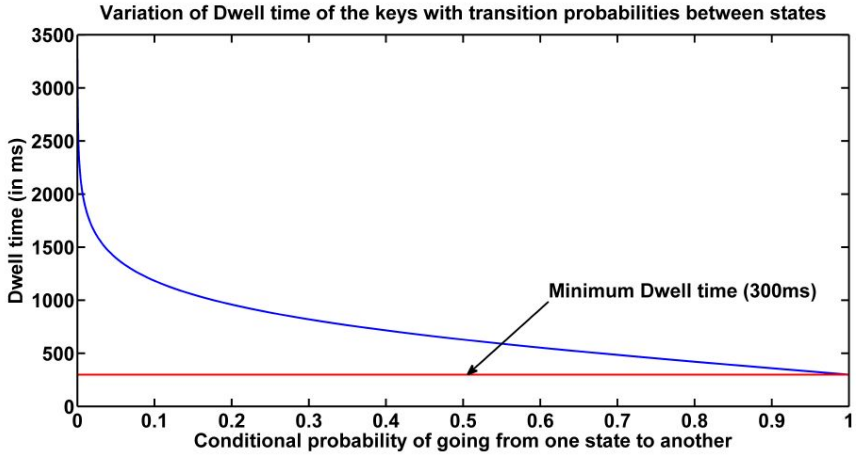


Figure 12. Variation of Dwell Time of Keys with Transition Probabilities Between Keys (States)

Figure 12 shows variation of dwell time with transition probabilities. With initial learning we expect user to learn at an exponential rate and to avoid problem of wrong selection, dwell time of keys should decrease at lesser rate as those keys approach lower dwell times (≈ 400ms). Choice of function (Equation 1) supports this claim as function flattens, as keys approach minimum dwell time (300ms). We keep starting dwell time of keys as maximum dwell time, 1000ms which can be manually adjusted by user. As maximum dwell time of any key should not exceed Dmax, Equation 1 is updated as:

$$D_i = \min(D_{max}, D_{min}[\log(\frac{2}{\mathrm{P}(i \mid j)} - 1) + 1]) \quad (2)$$

Updating Probability Matrix.

We consider movement of eye-gaze of user in the application like hops in a Markov chain from one state to another. At the start of application, a hop from one key to another is considered equally likely. In other words, conditional probability of hopping at any key 'i' from a key 'j' will be P (i | j) = $\frac{1}{n}$. Whenever user presses a key 'i' after key 'j', we update corresponding jth row of transition matrix. For example, when a user clicks key 2 after key 3 in a 3 key-layout, elements of transition matrix will be updated as:

$$\begin{bmatrix} \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{bmatrix} \rightarrow \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{4} & \frac{2}{4} & \frac{1}{4} \end{bmatrix}$$

Such an update increases probability of pressing key 2 after key 3 whereas simultaneously decreasing probability of pressing any other key. Since we follow an inverse relationship between dwell time and transition probability, dwell time of keys which are unlikely to be pressed is increased and decreases for likely keys. This helps in removing wrong selections of keys by users and enhances typing speed by faster dwell on likely keys.

Mathematical Evaluation of Algorithm.

We set starting dwell time of every key to 1000 ms which is maximum dwell time any key attains in our keyboard. As user starts using the keyboard, dwell time of each key changes accordingly with pattern of clicks (here click refers to selecting a key by dwelling at it for its dwell time). When comparing it with a keyboard with static dwell time for all keys, one can argue to keep static dwell time to be much lower than 1000 ms (maximum dwell time for our keys in our keyboard). However, since dwell time of keys are adaptive with use, one can show that, over time, our adaptive dwell keyboard will take much lesser time for task completion when compared to a static dwell keyboard.

Comparison of Task Completion Time in Static Dwell and Adaptive Dwell Keyboard.

As long as there is a pattern in using the application, total task completion time will be much lesser for an adaptive dwell-based keyboard (with our implemented algorithm) as compared to a static one. To verify the

For Review Only

same, let us assume two buttons on a screen (figure 13) which are clicked alternatively and repeatedly 'd' times where d ≥ 0. For example, d = 1 represents a user clicking Button 1 → Button 2.

We can compare total time for such a task with adaptive and static (non-adaptive) keyboard by considering the following variables:

- Dmax: Maximum dwell time any key attains on adaptive keyboard
- Dmin: Minimum dwell time any key attains on adaptive keyboard
- Dna: Static dwell time of all keys for non-adaptive keyboard
- ta: Duration of task after 'd' rounds for our adaptive keyboard
- tna: Duration of task after 'd' rounds for non-adaptive keyboard
- Total time taken in a non-adaptive keyboard after 'd' rounds, where Dna = Dmax:

$$t_{na} = 2 * D_{na} * d \tag{3}$$

- Entries of transition matrix for a two-button adaptive keyboard will change (after 'd' rounds) as:

$$\begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} \end{bmatrix} \rightarrow \begin{bmatrix} \frac{1}{3} & \frac{2}{3} \\ \frac{2}{3} & \frac{1}{3} \end{bmatrix} \rightarrow \begin{bmatrix} \frac{1}{4} & \frac{3}{4} \\ \frac{3}{4} & \frac{1}{4} \end{bmatrix} \cdots \begin{bmatrix} \frac{1}{d+2} & \frac{d+1}{d+2} \\ \frac{d+1}{d+2} & \frac{1}{d+2} \end{bmatrix}$$

The starting dwell time of Button 1 will be Dmax. Since dwell time for any key 'i' (Di) pressed after key 'j' in adaptive keyboard is calculated using Equation 2, total time taken in our adaptive keyboard after 'd' rounds will be calculated as:

$$t_a = D_{max} + 2 * D_{min} * \sum_{i=1}^{d} (\log(\frac{2(i+1)}{i} - 1) + 1) \tag{4}$$

$$\implies t_a = D_{max} + 2 * D_{min} * \sum_{i=1}^{d} (\log(\frac{i+2}{i}) + 1)$$

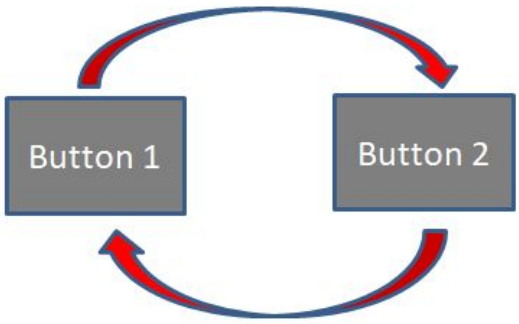


Figure 13. User Clicking Two Buttons Repeatedly

Equation 4 can be simplified further to:

$$t_a = D_{max} + 2 * D_{min} * [d + \log(\frac{(d+1)(d+2)}{2})]$$

It should be noted that here we assumed the time taken for visual search is negligible compared to time taken for a button click. It is assumed that user performs task repeatedly without pause. Figure 14 shows that time taken for task completion with our adaptive dwell-based keyboard will be always less compared to a static dwell-based keyboard over repeated use of application. This fact is established as the green curve is below all other curves after a few clicks. This is even possible when static dwell is kept 60 percent less than starting dwell time of keys for our adaptive keyboard.

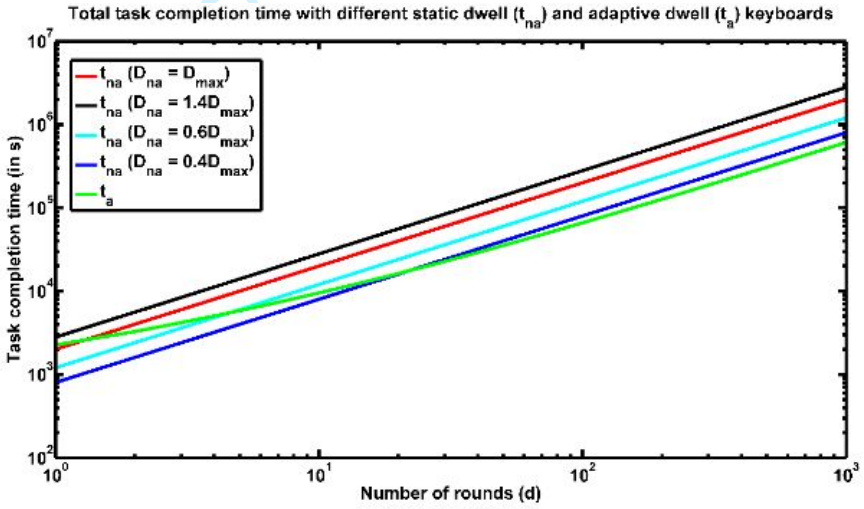


Figure 14. Comparing Total Task Completion Times with Different Static Dwell Time

Points where the green curve (time taken in adaptive keyboard with number of clicks) intersect other curves (representing static dwell-based keyboard set at different dwell) represent number of clicks for which tna = ta. Over time, as number of clicks (rounds) increases, total task completion time in adaptive dwell-based keyboard becomes much lesser in comparison to other keyboards with different static dwell settings.

6.1 User Study- With SSMI Users

We evaluated dwell time adaptation algorithm through a user study involving 16 participants, 9 able bodied users and 7 users with SSMI. The study is described in following subsections.

Participants.

Able-bodied users were university students (5 male, 4 female, average age: 29.5 years). Participants A, C, D, E, F, G, H (described in section 3) took part in the study.

Material.

We used a 17" display and an Intel NUC computer with dual core i5 processor and Windows 10 operating system. Display resolution was set to 1980 × 1080 pixels. Eye gaze was recorded using Tobii PC EyeX Mini eye tracker [65] with Tobii Software Development Kit. Frequency of eye tracking was 60 Hz and reported accuracy was 0.4º of visual angle [65].

Design.

As dwell time adaptation algorithm reduces dwell time for repetitive patterns, we set up a phrase to be typed three times. For able bodied users, the phrase was “I LOVE INTELLIGENT USER INTERFACE” while for users with SSMI, the phrase consisted of the word “PAPA”, considering their low typing speed. There were two conditions in the study, Non-Adaptive and Adaptive conditions – in Non-Adaptive condition, we used an orderly arrangement of letters with a fixed dwell time of 1000 ms; and in Adaptive condition, we used adaptive version of keyboard with dwell time adaptation algorithm with Dmax set to 1000 ms and Dmin to 300 ms. We measured average task completion time and total number of errors for both user groups. Task completion time was measured as time taken to type the abovementioned phrase. Since each participant types the target phrase three times, we measured three task completion times, hereafter referred as Sessions 1, 2 and 3. Total number of errors were measured as a summation of number of times backspace pressed on screen-1 and number of times back button pressed on screen-2. We did not use word completion or word prediction features for the user study.

Procedure.

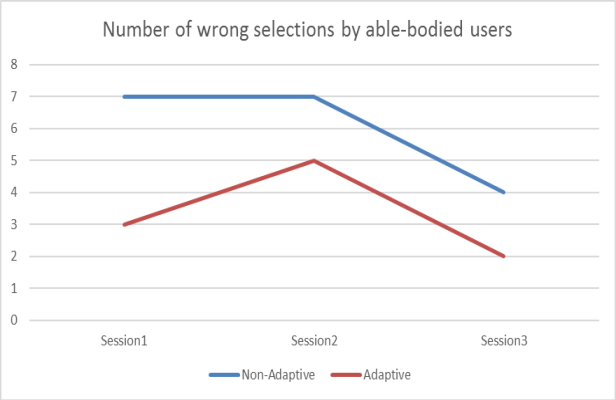
Initially, participants were briefed about purpose of study and had short practice session on both adaptive and non-adaptive versions of proposed keyboard. They undertook the trial with order of adaptive and non-adaptive conditions altered among participants to avoid order effect.

Results.

All 16 participants could complete the trial and did not report any discomfort while using the proposed system. We separately analyzed data for users with SSMI and their able-bodied counterparts. Average task completion time was analyzed using a Session (3) × Condition (2) ANOVA, followed by a pairwise t-tests for Session. Total number of errors was analysed using Chi-square test. For task completion times for able bodied users we found,

- Significant main effect of Session $F(2,16) = 17.499$, $p < 0.05$, $\eta 2 = 0.686$
- Significant main effect of Adaptation $F(1,8) = 13.723$, $p < 0.05$, $\eta 2 = 0.632$
- Interaction effect for Session and Adaptation was not found to be significant.

For Review Only



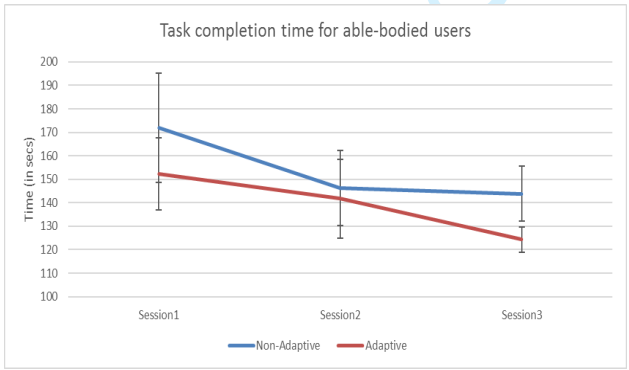
For *able-bodied users*., we noted that, average task completion times were statistically significantly different between all pairs of sessions at $p < 0.05$. We did not find any significant main or interaction effect (figures 15 and 16) for total number of errors for able-bodied users. However, it may be noted that, both average task completion times and total number of errors were reduced in session 3 for adaptive condition when compared to non-adaptive condition.

Figure 15. Comparing Average Task Completion Times of Able-Bodied Users

Figure 16. Comparing Total Number of Errors of Able-Bodied Users

For users with SSMI., we undertook a 2-way ANOVA and did not find either significant main effect or interaction effect for Session and Adaptation variables. However, we noted average task completion time for users with SSMI was 122.18 seconds in adaptive condition and 97.50 seconds in non-adaptive condition.

We further investigated adaptive layout for users with SSMI in greater details. It may be noted that they typed the phrase 'PA' six number of times. Figure 17 shows average typing time and total number of errors for each phrase. We noted that, dwell time progressively decreased after each round, and after Round 4 (tying PA for fourth time), there was an increase in average typing time as well as total number of errors (figure 17). However, standard deviation of typing time decreased till Round 4 and increased again in upcoming rounds.



Additionally, we noted overall typing speed for able-bodied users was 16.9 lpm and 6.6 lpm for users with SSMI without using any word completion or prediction features.

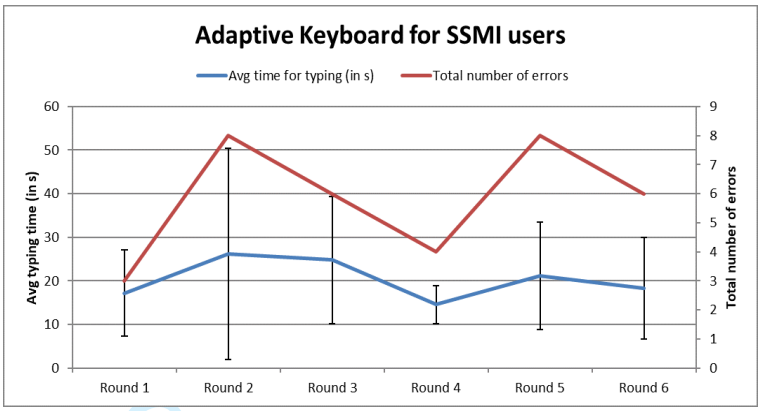


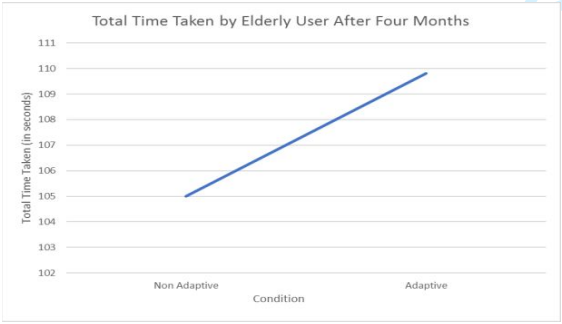
Figure 17. Typing Duration and Number of Errors for users with SSMI

6.2 User Study- With Expert User

In previous user study involving users with SSMI, all participants used the English virtual keyboard for the first time. Average task completion time for users with SSMI was higher in adaptive condition (122.18 seconds) than in non-adaptive condition (97.50 seconds). However, it may be noted that able bodied users typed faster in adaptive condition than in non-adaptive condition (Figure 15).

Earlier studies [4, 66] showed that, performance with human-computer interaction tasks like pointing and selection tasks significantly improve over time for users with SSMI. Besides the aforementioned user trial, we deployed the English virtual keyboard for personal use of an elderly gentleman. He is a renowned professor at authors' institute and on his 75th birthday, was diagnosed with ALS. We met him when he was 77 years old and by that time, he was already experiencing loss of body movement at all parts except for some uncontrolled head movement and his speech was slurred beyond recognition [73].

After four months, we compared users' typing speed for both non-adaptive and adaptive conditions. User typed the sentence "HOW MANY FINALISTS DO YOU HAVE" and took a total time of 105 seconds with 2.85 wpm in non-adaptive condition and 109.8 seconds with 3.27 wpm in adaptive condition (Figure 18) respectively.



For Review Only

Figure 18. Total Time Taken by Elderly User After Four Months

To further study the adaptation of Markov model-based algorithm, we analysed change in typing speed over time for several words on long sentences. We performed text mining on sentences recorded for four months and extracted most frequently used words. We calculated time taken to type each of most frequently used words over a period. For illustration purpose, we have selected four such words from the list of most frequently used words and plotted a graph (Figure 19) of time taken to type the words across four months. As expected by the algorithm, when user typed frequently, we observed that typing speed reduced over time with practice.

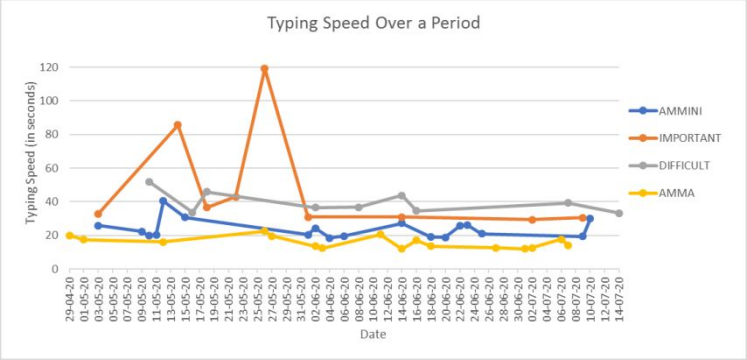


Figure 19. Typing Speed Over a Duration of Four Months

For calculating typing speed during regular use, we recorded a video (https://youtu.be/IGnYqVCHDY0?t=215) with him after four months of using our proposed English virtual keyboard. We analyzed the video and reported following events. Video starts with user already typed letters 'Yo' and the following table (table 7) shows users' activity.

Table 7: Results from Analysed Video Recorded with Expert User

Start Instance	End Instance	User Activity	Remark
00:01	04:00	User typed 'u can now directly deal with pradipta you can also forward the'	Typing Speed: • wpm: 3.01 • lpm: 15.56
04:00	04:14	User selected Backspace and resumed typing	Time spent: 14 secs
04:14	06:14	User typed 'Correspondence to Kamal whon'	Typing Speed: • wpm: 2.00 • lpm: 14.00
06:14	06:29	User selected Backspace and resumed typing	Time spent: 15 secs
06:29	08:23	User typed 'm I have mentioned in the men'	Typing Speed: • wpm: 3.68 • lpm: 15.26
08:23	08:25	User selected Backspace and resumed typing	Time spent: 2 secs

Review Only

08:25	08:40	User typed 'moir'	Typing Speed: • wpm: 4.00 • lpm: 16.00

Finally, the user typed following sentence "You can now directly deal with *anonymized* you can also forward the Correspondence to Kamal whom I have mentioned in the memoir". We noted that the video started with two letters ('Yo') already typed and calculated the speed of correctly typing as:

- Correct wpm: 2.54
- Correct lpm: 13.99

It may be noted that, we neither instructed user about what to type as in the previous user study with SSMI users nor instructed to type as fast as he could. Further, we noted that after pressing Blank space, textbox started to show sentence from beginning and wasted time of user to resume typing. Without time involved in error recovery, typing speed increased to 2.70 wpm and 14.88 lpm. It is only 12% slower than our able-bodied participants reported in previous user study and 2.25 times faster than first time users with SSMI.

6.3 Discussion

This study compared adaptive dwell time adjustment algorithm for users with SSMI and able-bodied participants. The algorithm significantly increased typing speed for able bodied users. While for users with SSMI, it may be noted that the short duration of our trial, inexperience, and variable range of abilities of users with SSMI impeded us to record a higher typing speed. We could not train all users for longer duration like Majaranta's study [6, 43], which motivated users to perform the task by giving prizes for fastest typing and higher typing speeds were achieved after the 10th session (day). However, for a single user, typing speed indeed improved in adaptive condition after four months of practice.

We could not render all keys in a single screen and our users had more severe physical impairment than participants involved in Mott's study [46]. Unlike OptiDwell [45] which operates on discrete states for updating dwell time, our model updates dwell time in a continuous state, which means our model takes continuous real values for dwell time between Dmax and Dmin, reducing the chance of sudden peaks in dwell time of keys. Due to shorter duration of trial, the Markov model did not get enough time to converge to true values of dwell times for all keys.

It may be noted from figure 17 that, number of errors increased and typing speed decreased after Round 4, when dwell time decreased. In future studies, we can personalize the selection of Dmax and Dmin like Majaranta's study, although the process may involve multiple iterations with users. From figure 12, it may be noted that even if we select Dmax higher than a fixed value of dwell time, over prolonged use, overall typing speed will improve in adaptive version. Presently, we are training users to operate the keyboard and hoping to report a higher typing speed than the first-time use reported in this study.

7 OVERALL DISCUSSION

This paper aims to develop a virtual keyboard interface for English and other Indian languages and to optimize dwell time for selection operation by users with SSMI. Eye gaze-controlled interface has a long history, perhaps started with Zhai's [16] pioneering work on MAGIC system and Majaranta [6, 43] and Hollomon [67] presented

For Review Only

a detailed history of such interfaces by classifying them into explicit, attentive, and passive interfaces. However, Farrell [68] noted that “humans use their eyes naturally as perceptive, not manipulative, body parts.” while Holmqvist [69] reported that gaze-controlled interfaces provide “new opportunities to communicate, interact and perform activities independently, as long as conditions are right”. Researchers already investigated visual function in children with SSMI and reported presence of nystagmus in less than 10% in one representative sample [70] and about 72% in another sample [70]. Nystagmus was accompanied by loss of visual acuity, contrast sensitivity, strabismus and Fazzi [71] reported that “clinical expression of cerebral visual impairment can be variable” requiring a case-by-case analysis of end users. This wide range of differences in terms of ocular parameters make many existing interfaces hard to use by users with SSMI. Developing eye gaze-controlled interface and virtual keyboard is challenging for these users due to following constraints:

- They cannot fixate attention equally well for all positions on a computer screen
- They require a minimum size of target and inter-element spacing among different screen elements due to uncontrolled eye gaze movement or nystagmus. Presence of nystagmus, subsequent minimum target size and inter-element spacing makes approaches like Dwell Free typing [70] infeasible for this user group
- They did not use any electronic virtual keyboard before and need training to use any new software
- English is their second language, and they require a keyboard in their first language.

Initially, we developed an application for comparing accessibility of different screen regions for an eye gaze-controlled interface. The algorithm developed helps in finding optimal screen area that can be used to organize on screen elements. We noted that, unlike able bodied users, most users with SSMI found difficulty to select screen elements placed on top left corner of screen in a gaze-controlled interface, which was also confirmed by earlier publication [48].

Based on these results, we proposed to design an optimized gaze-controlled virtual keyboard for English and Kannada languages. We developed a two-level and three- level tree structured layout and used Genetic algorithm to optimize the arrangement of letters in screen elements of the layout, respectively. Unlike the traditional method of defining objective function based on frequency of letters, bigram, trigram or any n-gram model, our objective function is defined as total eye gaze movement required to type a particular corpus. Hence, objective function can adapt to different corpus based on language and context of use.

Earlier, Karrenbauer [50] used integer programming for letter assignment problem for any given keyboard layout using bi-gram frequency. It can be argued to use trigram or even higher order n-gram frequency that may eventually complicate formulating the objective function. Further, using a Markov Model, we optimized the dwell time irrespective of language model to adapt it easily for resource-constrained Indian languages. Finally, we then proposed an optimized three-level gaze-controlled keyboard for Kannada language which can be extended to other Indian languages like Hindi, Tamil, Telugu, Malayalam by just replacing the character set, requiring no change in design of layout.

User studies with able bodied users significantly increased typing speed compared to a non-adaptive version while for users with SSMI, the difference was not statistically significant but higher for adaptive version. For user studies involving participants with SSMI, we must consider a two screen-layout as control condition, as these users could not type a single letter using existing Windows UWP full screen keyboard. We reported that a single participant could type faster than first time users after 4 months of practice with the keyboard.

For Review Only

For the cross-sectional study (Study 1), we used same D_{max} and D_{min} values of the dwell time adaptation algorithm for all participants for internal validity so that we can use statistical testing by taking average typing time of all participants. The length of experimental phrase (only one word three times) was limited by the availability of individual participant with SSMI, who were first time user of the virtual keyboard got tired and bored easily. Fir the longitudinal study involving a single user with ALS, we allowed adjusting the D_{max} and D_{min} values and he was able to type sentences as long as he wanted.

In terms of speed of typing it may be noted that the reported typing speed is achieved without any word completion or prediction features. The accuracy and latency of next letter or word prediction feature depends on data structure, particular retrieval algorithm and actual usage of the keyboard. We did not want to confound the result with letter or word prediction features and hence reported typing speed only with the two-layer layout and dwell time adaptation. However, our keyboard is integrated with a Trie based word completion and next word prediction feature, that can further increase typing speed (Figure 20). We have limited space in terms of spacing predicted letter or word, which will also affect typing speed.

We do not claim it to be the fastest keyboard either. Our particular user group could not use existing virtual keyboards, an example of the struggle can be seen from 2:11 mins to 2:25 mins of the demo video at https://youtu.be/IGnYqVCHDY0?t=131 We also got similar results with other users who could not type a single word correctly with existing virtual keyboards. Cerebral Palsy or ALS can result in wide variation in range of abilities and we described range of abilities of users in section 3. It may not be ideal to compare typing speeds of keyboards without considering range of abilities of users. For example, Mott [46] reported results with 5 users with ALS and our expert participant also had ALS. However, we recorded data after 5 years of onset of ALS and by that time the participant could neither keep his head straight and nor talk [page 295 of ref 72], while Mott [46] did not report in details the condition of their participants.

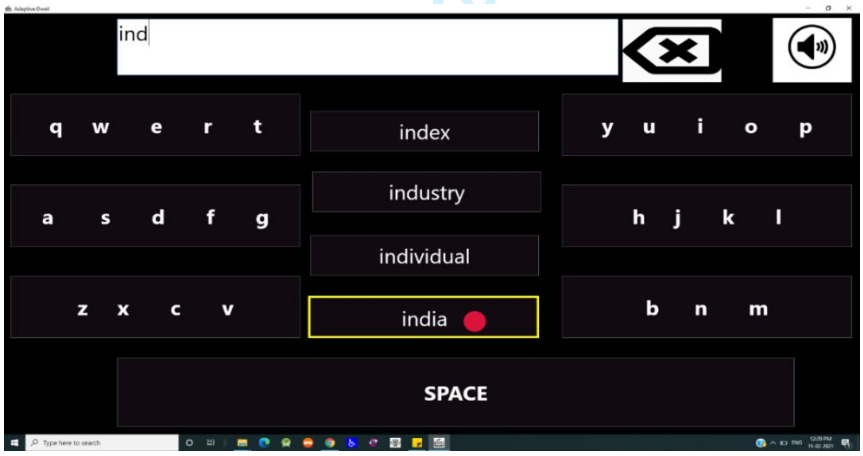


Figure 20. Virtual Keyboard with Word Prediction

A web implementation of the virtual keyboard can be viewed at http://cambum.net/VirtualKeyboard/ , it requires 1366 × 768 or higher resolution for rendering. As the keyboard has only limited number of screen elements, it can also be used by webcam based eye trackers which are less accurate than commercial infrared based trackers, a demonstration video can be seen at https://youtu.be/2pyX6O2xTFw?t=17. The same

application can be used to send messages to another participant through an online messaging application by choosing between two options upon clicking on a picture showing speak and send buttons. The keyboard is modular, in the sense that, the optimized layout is independent of dwell time adaptation algorithm and any combinations between these two algorithms can be offered – for example, for a QWERTY keyboard user, we can only deploy the dwell time adaptation algorithm with QWERTY layout while for a regional language keyboard we can offer both optimized layout and dwell time adaptation.

8 CONCLUSION

This paper presents an eye gaze controlled adaptive virtual keyboard for users with SSMI that had screen elements at preferred position of users maintaining a minimum target size and inter-element spacing. The keyboard was implemented in English language and evaluated for both users with SSMI and their able-bodied counterparts. The proposed layout is optimized with respect to eye gaze movement distance and the dwell time is adapted for typing common patterns. Further, we proposed an optimised gaze controlled virtual keyboard layout for Kannada language which can be extended for other Indian languages like Telugu, Tamil, Malayalam, Hindi and so on. The adaptive system increased typing speed statistically significantly for able bodied users than a non-adaptive version. A case study with an elderly participant with SSMI found a typing speed of 2.70 wpm and 14.88 lpm after 4 months of practice without using word completion or prediction features.

For Review Only

REFERENCES

[1] Biswas P., Joshi R., Chattopadhyay S., Acharya U. R., and Lim T. C., 2015, Interaction Techniques for Users with Severe Motor-Impairment, A Multimodal End-2-End Approach to Accessible Computing, Springer-Verlag London.

[2] Faller, J., Scherer, R., Costa, U., Opisso, E., Medina, J., & Müller-Putz, G. R., 2014, A co-adaptive brain-computer interface for end users with severe motor impairment. PloS one, 9(7), e101168.

[3] Lazarou, I., Nikolopoulos, S., Petrantonakis, P. C., Kompatsiaris, I., & Tsolaki, M., 2018, EEG-based brain–computer interfaces for communication and rehabilitation of people with motor impairment: a novel approach of the 21st century. Frontiers in human neuroscience,

[4] Borgestig, M., Sandqvist, J., Parsons, R., Falkmer, T., & Hemmingsson, H., 2016, Eye gaze performance for children with severe physical impairments using gaze-based assistive technology—A longitudinal study. Assistive technology, 28(2), 93-102.Brouillette.

[5] Eye-gaze control technology, Available at https://www.cerebralpalsy.org.au/about-cerebral-palsy/interventions-andtherapies/eye-gaze-technology-for-children-and-adults-with-cerebralpalsy/#1473 737787180-c2ceacf9-ff87, Accessed on 8 November 2017.

[6] Majaranta, P., Aoki, H., Donegan, M., Hansen, D. W., Hansen, J. P., Hyrskykari, A.,& Räihä, K., 2012, Gaze Interaction and Applications of Eye Tracking: Advances in Assistive Technologies (pp. 1-382). Hershey, PA: IGI Global.doi:10.4018/978-1-61350-098-9.

[7] Duchowski A. T., 2007, Eye Tracking Methodology. Springer.

[8] Stephanidis, C. et. al., Designing Human Computer Interfaces for Quadriplegic People, 2003, ACM Transactions on Computer-Human Interaction, pp 87-118,10(2).

[9] Communicate with the world using the power of your eyes, Available at http://www.eyegaze.com/eye-tracking-assistive-technology-device/,Accessed on 8 September 2017.

[10] Virtual/On-screen keyboard, Available at https://ds.gpii.net/learn/accessibility-masterlist/virtual-screen-keyboard accessed on 16 July 2019.

[11] Disabled Online, Available at http://www.disabledonline.com/products/disabledonline-store/switches/sensitive-switches/soft-switch/,accessed on 2 August 2019.

[12] Meena, Y. K., Cecotti, H., Wong-Lin, K., Dutta, A., & Prasad, G., 2018, Toward optimization of gaze-controlled human–computer interaction: application to hindi virtual keyboard for stroke patients. IEEE Transactions on Neural Systems and Rehabilitation Engineering, 26(4), 911-922.

[13] Meena, Y. K., Chowdhury, A., Sharma, U., Cecotti, H., Bhushan, B., Dutta, A., & Prasad, G., 2018, A hindi virtual keyboard interface with multimodal feedback: a case study with a dyslexic child. In 2018 32nd British human computer interaction conference (BHCI).

[14] Ward, D. J., Blackwell, A. F., & MacKay, D. J., 2000, Dasher-a data entry interface using continuous gestures and language models. In UIST (pp. 129-137).

[15] Li, Y., Chen, L., & Goonetilleke, R. S., 2006, A heuristic-based approach to optimize keyboard design for single-finger keying applications. International Journal of Industrial Ergonomics, 36(8), 695-704.

[16] Zhai, S., Hunter, M., & Smith, B. A., 2000, The Metropolis Keyboard??? An Exploration of Quantitative Techniques for Virtual Keyboard Design. In in UIST'2000: Proceedings of the 13th annual ACM symposium on User interface software and technology.

[17] Joshi, A., Dalvi, G., & Joshi, M., 2014, Corpus of Marathi Word Frequencies from Touch-Screen Devices Using Swarachakra Android Keyboard. In Proceedings of the India HCI 2014 Conference on Human Computer Interaction (p. 74). ACM.

[18] Huckauf, A., & Urbina, M. H. (2008, March). Gazing with pEYEs: towards a universal input for various applications. In Proceedings of the 2008 symposium on Eye tracking research & applications (pp. 51-54).

[19] Lau, S. F., & Chan, H. C., 2017, A special virtual keyboard for disabled computer users. Lecture notes in engineering and computer science.

[20] Patidar, P., Raghuvanshi, H., & Sarcar, S. (2014). Quickpie: an interface for fast and accurate eye gazed based text entry. arXiv preprint arXiv:1407.7313.

[21] Prabhu, V., & Prasad, G., 2011, Designing a virtual keyboard with multi-modal access for people with disabilities. In 2011 World Congress on Information and Communication Technologies (pp. 1133-1138). IEEE.

[22] Raynal, M., & Vigouroux, N., 2005, Genetic algorithm to generate optimized soft keyboard. In CHI'05 extended abstracts on Human factors in computing systems (pp. 1729-1732). ACM.

[23] MacKenzie, I. S., & Zhang, X., 2008, Eye typing using word and letter prediction and a fixation algorithm. In Proceedings of the 2008 symposium on Eye tracking research & applications (pp. 55-58). ACM.

[24] Wagner, M. O., Yannou, B., Kehl, S., Feillet, D., & Eggers, J., 2003, Ergonomic modelling and optimization of the keyboard arrangement with an ant colony algorithm. Journal of Engineering Design, 14(2), 187-208.

[25] Kim, H., Yi, S. and Yoon, S.Y., 2019. Exploring touch feedback display of virtual keyboards for reduced eye movements. Displays, 56, pp.38-48.

[26] Indian Languages, Available at https://www.britannica.com/editor/The-Editors-of-Encyclopaedia-Britannica/4419, assessed on 20 April 2020.

[27] Dravidian Languages, Available at https://en.wikipedia.org/wiki/Dravidian_languages, assessed on 20 April 2020.

[28] Indic onscreen Keyboard iGoogle gadgets, Available at http://www.google.com/igMicrosoft 2009 accessed on 18 August 2018.

[29] Onscreen keyboard, Available at http://windows. microsoft.com/en/windows7/Type-without-using-the-keyboard-On-ScreenKeyboard accessed on 18 August 2018.

[30] Online onscreen virtual Hindi keyboard emulator on the Internet, Available at http://www.gate2home.com/?language=hi, accessed on 18 August 2018.

[31] Indian languages virtual keyboard, Available at http://www.keyboard4all.com, accessed on 18 August 2018.

[32] A predictive text input system, Available at http://www. lipik.in, accessed on 18 August 2018

[33] Hindi virtual keyboard, Available at http://www.mylanguages.org/hindi_keyboard.php, accessed on 18 August 2018.

[34] Technology Development for Indian Languages, Available at Keyboard standard, 1986, http://www.tdil.mit.gov.in/pdf/KeyboardStandard2.pdf accessed on 20 April 2020.

[35] Malas, T. M., Taifour, S. S., & Abandah, G. A., 2008, Toward optimal Arabic keyboard layout using genetic algorithm. Proc. 9th Int'l Middle Eastern Multiconf. on Simulation and Modeling (MESM 2008), 50-54.

[36] Khan, M. A., & Ali, M. N., 2009, Design of Urdu Virtual Keyboard.): 'Book Design of Urdu Virtual Keyboard', 126-130

[37] Brouillette, A., Sarmah, D., & Kalita, J., 2012, Multi-objective optimization for efficient brahmic keyboards. In Proceedings of the Second Workshop on Advances in Text Input Methods (pp. 29-44).

[38] Cecotti, H. A multimodal gaze-controlled virtual keyboard. 2016, IEEE Transactions on Human-Machine Systems, 46(4), 601-606.

[39] Rajanna, V., & Hansen, J. P. (2018, June). Gaze typing in virtual reality: impact of keyboard design, selection method, and motion. In Proceedings of the 2018 ACM Symposium on Eye Tracking Research & Applications (pp. 1-10).

[40] Morimoto, C. H., Leyva, J. A., & Diaz-Tula, A. (2018, June). Context switching eye typing using dynamic expanding targets. In Proceedings of the Workshop on Communication by Gaze Interaction (pp. 1-9).

[41] Tuisku, O., Majaranta, P., Isokoski, P., & Räihä, K. J., 2008, Now Dasher! Dash away: longitudinal study of fast text entry by Eye Gaze. In Proceedings of the 2008 symposium on Eye tracking research & applications (pp. 19-26). ACM.

[42] Pi, J., & Shi, B. E. (2017, July). Probabilistic adjustment of dwell time for eye typing. In 2017 10th International Conference on Human System Interactions (HSI) (pp. 251-257). IEEE.

[43] Majaranta P. and Räihä K. J., 2002, Twenty years of eye typing: systems and design issues. In Proceedings of the 2002 symposium on Eye tracking research & applications (ETRA '02). ACM, New York, NY, USA, 15-22.

[44] Penkar A.M., Lutteroth C. and Weber G., Designing for the Eye – Design Parameters for Dwell in Gaze Interaction, 2012, OZCHI.

[45] Nayar A. et al., OptiDwell: Intelligent Adjustment of Dwell Click Time, 2017, Proceedings of International Conference on Intelligent User Interfaces.

[46] Mott, M. E., Williams, S., Wobbrock, J. O., & Morris, M. R., 2017, Improving dwell-based gaze typing with dynamic, cascading dwell times. In Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems (pp.2558-2570). ACM.

[47] Feit, A. M., Williams, S., Toledo, A., Paradiso, A., Kulkarni, H., Kane, S., & Morris, M. R., 2017, Toward everyday gaze input: Accuracy and precision of eye tracking and implications for design. In Proceedings of the 2017 chi conference on human factors in computing systems (pp. 1118-1130). ACM.

[48] Jeevithashree, D. V., Saluja, K. S., & Biswas, P., 2019, A case study of developing gaze-controlled interface for users with severe speech and motor impairment. Technology and Disability, 31(1-2), 63-76.

For Review Only

[49] Hornof A., Cavender A. and Hoselton R., 2004, EyeDraw: A System for Drawing Pictures with Eye Movement, Proceedings of ASSETS'04, Atlanta, Georgia, USA.

[50] Karrenbauer A, Oulasvirta A., 2014, Improvements to keyboard optimization with integer programming. Proceedings of the 27th annual ACM symposium on User interface software and technology, 621–626.

[51] Marcus, M., Santorini, B., & Marcinkiewicz, M. A., 1993, Building a large, annotated corpus of English: The Penn Treebank.

[52] English Alphabets, Available at https://en.wikipedia.org/wiki/English_alphabet accessed on 02 december 2020.

[53] Kannada/Alphabet, Available at https://en.wikibooks.org/wiki/Kannada/Alphabet accessed on 12 March 2020.

[54] Naveena, C., & Aradhya, V. M., 2012, Handwritten character segmentation for kannada scripts. In 2012 World Congress on Information and Communication Technologies (pp. 144-149). IEEE.

[55] AGuide to Hindi Language, Available at http://www.bbc.co.uk/languages/other/hindi/guide/alphabet.shtml#:~:text=The%20standard%20Hindi%20alphabet%2C%20as,as %20consonants%20in%20standard%20Hindi, accessed on 16 September 2020.

[56] Tamil Alphabets, Available at http://tamilcube.com/learn-tamil/tamilalphabets-chart.aspx accessed on 20 February 2020.

[57] TamilLnguage/Letters, Available at https://en.wikiversity.org/wiki/Tamil_Language/Letters, accessed on 25 February 2020.

[58] TeluguLanguage, Available at https://en.wikipedia.org/wiki/Telugu_language#Phonology accessed on 13 March 2020.

[59] Samanta, D., Sarcar, S., & Ghosh, S., 2013, An approach to design virtual keyboards for text composition in Indian languages. International Journal of Human-Computer Interaction, 29(8), 516-540.

[60] ConjunctConsonants, Available at http://www.learningtelugu.org/conjunctconsonants.html accessed on 13 March 2020.

[61] Kannada monolingual sample dataset, Indian Language Technology Proliferation and Development Centre, http://www.tdil-dc.in/index.php?option=com_download&task=fsearch&Itemid=547&lang=en, accessed on 20 October 2019.

[62] Phonics teaching step-by-step, Available at https://www.theschoolrun.com/phonics-teaching-step-by-step, accessed on 21 June 2020.

[63] Introduction to Phonics, Available at https://www.literacyideas.com/howtoteachphonics#::text=Decoding%20and%20Encoding%3A%20The%20Bridge,skill%20of%20writing%2C%20or%20encoding, accessed on 21 June 2020.

[64] Microsoft Indic Language Input Tool (ILIT) 2020, Available at https://www.microsoft.com/en-in/bhashaindia/downloads.aspx, Accessed on 1 January 2020.

[65] Tobii PCEyeXMini Tracker, Available at https://www.tobiidynavox.com/devices/eye-gaze-devices/pceye-miniaccess-windows-control/#Specifications, Accessed on 23 March 2018.

[66] Sharma VK, Murthy LRD, Saluja KPS, Mollyn V, Sharma G and Biswas P, 2020, Webcam Controlled Robotic Arm for Persons with SSMI, Technology and Disability 32 (3).

[67] Hollomon M. J., Kratchounova D., Newton D. C., Gildea K. and Knecht W. R.,2017, Current Status of Gaze Control Research and Technology Literature Review, Report of Civil Aerospace Medical Institute Federal Aviation Administration, Oklahoma City, OK 73125.

[68] Farrell S., Zhai S., 2005, System and method for selectively expanding or contracting a portion of a display using eye-gaze tracking, US Patent No.:20050047629 A1.

[69] Holmqvist E., Thunberg G. & Dahlstrand M. P., 2017, Gaze-controlled communication technology for children with severe multiple disabilities: Parents and professionals' perception of gains, obstacles, and prerequisites, Assistive Technology.

[70] Vasanth J., Jacob N. and Viswanathan S., 2014, Visual Function Status in Children with Cerebral Palsy, Optometry & Visual Performance 2(3), pp 251-253.

[71] Fazzi E. et al, 2007, Spectrum of Visual Disorders in Children with Cerebral Visual Impairment, Journal of Child Neurology 22 (3), pp. 294 – 301.

[72] Kristensson P.O. and Vertanen K., 2012, The Potential of Dwell-Free Eye-Typing for Fast Assistive Gaze Communication, Proceedings of the ACM International Symposium on Eye Tracking and Research

[73] Vijayan M., A life along men, women and Molecules, Indian National Science Academy, ISBN 978-81-939482-6-2

For Review Only